UNIVERSITY OF HERTFORDSHIRE

DEPARTMENT OF COMPUTER SCIENCE

Beyond League Tables: A K-Means Clustering Analysis of Institutional Value and Graduate Outcomes in the London University Market

BSc HONOURS COMPUTER SCIENCE (APPLIED DATA SCIENCE) (ONLINE)

ABIGAIL OKOI-OBULI | STUDENT ID - 24128739

6WCM2016 – DATA SCIENCE PROJECT (COM)

FINAL PROJECT REPORT | April 2026

1.0 ACKNOWLEDGEMENT

To my mother and sisters, who have supported me every step of this journey, you have my warmest thanks. To my father, brother and youngest sister halfway across the world, every word of encouragement was deeply appreciated.

To my project supervisor, Dr Peter Lane, I am incredibly grateful for your advice and support, as this project would not be possible without it.

To the person I have and am still becoming by working each day diligently, be proud of your accomplishments.

And last but certainly not least, the greatest thanks belongs to God.

2.0 ABSTRACT

This project aims to complement traditional university league tables by evaluating the institutional value and graduate outcomes of twenty-two London-based Higher Education Institutions (HEIs). While conventional rankings often over-emphasize research output and reputation, potentially obscuring institutions that deliver exceptional practical value and student satisfaction, this study provides prospective applicants with a multi-dimensional, data-driven framework to identify universities that align with specific career and academic priorities.

The methodology integrated diverse datasets, including National Student Survey (NSS) satisfaction metrics, Higher Education Statistics Agency (HESA) student data, Discover Uni Employment and Entry data, as well as global prestige rankings like QS and THE. Using Python, a K-means clustering algorithm was applied to categorize institutions based on a custom Course Value Index, employment success, and academic prestige. The results were translated into ten Tableau visualizations and a dashboard, featuring scatter plots with cluster-specific trend lines, box plots for consistency validation, and a dynamic comparison dashboard. The source code is provided in Appendix A, along with the link to the entire Tableau Workbook.

3.0 TABLE OF CONTENTS

1.0 ACKNOWLEDGEMENT 2

2.0 ABSTRACT 3

3.0 TABLE OF CONTENTS 4

4.0 INTRODUCTION 7

4.1 Project Aim 7

4.2 Objectives 7

4.3 Summary of Chapters 8

4.5 Glossary 9

4.6 Ethics Approval Statement 9

4.7 Artificial Intelligence Statement 9

5.0 LITERATURE REVIEW 10

5.1 Limitations of Traditional Rankings 10

5.2 Clustering as a Tool for Evaluation of Higher Education Institutions. 10

5.3 Global Applications of K-Means Clustering for University Evaluations and the London Gap 10

6.0 REQUIREMENTS 12

6.1 Methodology 12

6.2 Functional Requirements 12

6.3 Non-Functional Requirements 12

6.4 Legal, Ethical, Environmental and Professional Constraints 13

7.0 DESIGN 14

7.1 System Architecture 14

7.2 Algorithmic Design: K-Means Implementation 14

7.3 The Interactive Dashboard 14

7.4 Data Structure Design 15

8.0 TECHNICAL DEVELOPMENT 16

8.1 Development Environment 16
8.2 Project Plan and Execution 16
8.3 Data Analysis and Pipeline Design Decisions 18
8.4 Handling Uncertainties and Requirements Resolution 18
8.5 Discussion of Problems and Risk Mitigation 19
8.6 Progressive Refinement: Implementing K-Means 20
9.0 PROJECT EVALUATION 21
9.1 Evaluation Strategy 21
9.2 Algorithmic Evaluation (Summative Model Testing) 21
9.3 Usability Evaluation 21
9.4 Requirements Traceability Evaluation 22
10.0 RESULTS OF EVALUATION 23
10.1 Algorithmic Evaluation Results 23
10.2 Usability Evaluation Results 25
10.3 Requirements Traceability Evaluation Results 25
11.0 DISCUSSION 27
11.1 The Anatomy of the London Market: Decoding the Archetypes 27
11.2 The Prestige-Tariff Paradox and Gatekeeping 29
11.3 Subject-Level Sensitivity and Institutional Quality 29
11.4 Critical Analysis of Graduate Outcomes and Diversity 30
11.5 The Impact of Prestige on Student Satisfaction 30
11.6 Spatial Dynamics: The Centrality of International Appeal 31
11.7 Ranking Dissonance in Traditional League Tables 33
11.8 Dynamic Dashboard for Customizable Comparisons 35
11.9 Methodological Limitations 36
12.0 CONCLUSION 37
12.1 Achievement of Objectives 37
12.2 Reflection on Limitations and Failed Objectives 37
12.3 Personal and Professional Reflection 38

12.4 Future Work 38
12.5 Final Summary 38
13.0 BIBLIOGRAPHY 40
14.0 APPENDICES 41

4.0 INTRODUCTION

The selection of a Higher Education Institution (HEI) is one of the most significant financial and professional decisions an individual makes. However, the current landscape of university evaluation is dominated by linear league tables that often prioritize institutional heritage and research output over granular student outcomes and academic value, failing to capture the nuance of best-fit.

This project aims to dismantle the prestige bias of traditional rankings by applying unsupervised machine learning to identify distinct institutional archetypes. By developing an interactive, data-driven dashboard, this project empowers prospective applicants to evaluate London's universities based on a multi-dimensional analysis of student satisfaction, career success, and academic value.

4.1 Project Aim

The primary aim of this project is to develop a clustering-based analytical tool that categorizes twenty-two universities in London into value groups, providing a transparent alternative to traditional rankings. Using K-Means clustering, it will analyze key metrics such as university ranking, student satisfaction, entry requirements, and more, to group universities into distinct clusters that reflect their strengths and weaknesses more accurately than a standard hierarchical ranking.

The project will encompass only select universities based in London, focusing on their undergraduate data for the analysis. Thus, specialist and postgraduate institutions have been excluded from this study to reflect these aims.

4.2 Objectives

To fulfil this aim, the following core objectives were established:

- Interactive Metric Visualization: Develop a dashboard capable of displaying and comparing metrics including global rankings, student satisfaction, and employment outcomes.
- Subject-Level Granularity: Implement functionality to drill down into the aforementioned metrics at the individual course or subject level where the data permits.
- Comparative Analysis Engine: Enable the direct comparison of different universities based on custom-weighted academic and career metrics.

- The Course Value Index: Design and implement a composite score to calculate "potential value added" for each institution and course.
- Cluster Discovery: Utilize K-Means clustering to identify value archetypes, such as those offering high employment outcomes despite lower entry requirements.

Secondary objectives for this project are outlined as follows:

- Demographic Mapping: Including the ratio of international-to-home students to identify global hubs.
- Academic Attainment Mapping: Mapping the proportions of degree classifications (First, Second, and Third-class degrees).
- Operational Metrics: Visualizing institutional scale and spatial distribution across London boroughs.

4.3 Summary of Chapters

This project is organized into nine chapters, following a logical progression from theoretical grounding to technical execution and critical reflection:

- **Literature Review -** synthesizes current scholarship on the flaws of linear rankings and the mathematical validity of using K-means clustering for institutional evaluation.
- **Requirements Specification** - details the functional and non-functional requirements, including a framework for Legal, Ethical, Professional, and Environmental (LEEP) issues.
- **Technical Design** - outlines the system architecture, the data processing pipeline (Python), and the user interface design (Tableau).
- **Technical Development** - documents the practical implementation, including data cleaning and preprocessing, creating Prestige Score and Course Value Indexes for clustering and the strategic refinement of the project scope to focus on the 22 London higher institutions.
- **Evaluation Methodology** - establishes the strategy for validating both the mathematical accuracy of the clusters and the usability of the interface.
- **Results** - reports the empirical outputs of the evaluation, including the Elbow Method values (k=1 to k=10) and system performance metrics.

- **Discussion** interprets the data visualizations and dashboard to provide a deep-dive analysis of the discovered archetypes, the "Prestige-Tariff Paradox," and the impact of geography on internationalization.
- **Conclusion and Reflection** - offers a self-critical assessment of the project's successes and an honest reflection on the objectives that were modified due to data limitations.

4.5 Glossary

- **Course Value Index** – a custom attribute that evaluates a specific course based on three distinct pillars: graduate outcomes, student satisfaction, and institutional reputation.
- **Developing/ Entry Level** – one of the clusters, denoted by blue.
- **Elite Academic Tier** – one of the clusters, denoted by orange.
- **HE** – Higher Education
- **HEI** – Higher Education Institutions
- **HESA** – the Higher Education Statistics Agency
- **High-Value Career Hubs** – one of the clusters, denoted by red.
- **KISCOURSEID** – course-specific identification number.
- **NSS** – National Student Survey
- **NSSSBJ** – attribute containing Common Aggregation Hierarchy (CAH) subject-level data
- **Prestige Score** - calculates an institution's prestige by normalizing and inverting it's QS Score.
- **Specialist & Niche** – one of the clusters, denoted by green.
- **UKPRN –** UK Register of Provider Reference Number, unique ID assigned to each higher institution.

4.6 Ethics Approval Statement

This project does not require ethics approval as it utilizes data that has been made available for public use by HESA and Discover Uni.

4.7 Artificial Intelligence Statement

Artificial intelligence has not been used for the creation of this project, and was only used to correct grammatical errors in line with the university's regulations guiding the use of AI for academic purposes.

5.0 LITERATURE REVIEW

5.1 Limitations of Traditional Rankings

Prospective students often use university rankings such as Quacquarelli Symonds(QS) or Times Higher Education (THE) when deciding what higher education institution to apply to or attend. While popular for their simplicity, linear rankings do not fully encapsulate the differences between institutions, as well as the suitability for a given student (Poole, et al., 2018).

Their work suggests that traditional league tables often fail to address the "best fit" for individual students, emphasizing that institutional marketing often prioritizes prestige over applicants' specific needs and aspirations. This creates a vacuum where data-driven tools could be used to help bridge the gap between reputation and personal value.

5.2 Clustering as a Tool for Evaluation of Higher Education Institutions.

To move beyond the limitations of one-dimensional hierarchical lists, recent scholarship has turned to unsupervised machine learning, more specifically cluster analysis, to uncover hidden institutional archetypes. Matlis, et al. (2024) demonstrate that clustering analysis allows for university groupings to be unveiled that would otherwise be obscured by standard academic rankings. By grouping institutions based on shared characteristics rather than a singular score, their work provides a precedent for identifying peer groups that compete on specific metrics like research output or teaching quality, rather than just brand recognition (Matlis, et al., 2024).

5.3 Global Applications of K-Means Clustering for University Evaluations and the London Gap

Across the world, there has been a shift toward using clustering, particularly in diverse or emerging education systems where standardized rankings may be less effective. Al-Hagree et al. (2023) successfully applied cluster analysis to Yemeni universities, proving that even in varied economic contexts, algorithmic grouping can identify top performers that traditional metrics might miss.

Building on this, Kucukdeniz et al. (2025) advocate for a specific K-means approach to evaluate research universities in emerging systems. Their findings suggest that K-means is particularly robust for handling the multi-dimensional data inherent in higher education, such as the mix of satisfaction, enrollment, and outcomes used in this study, which provides a rigorous mathematical foundation for moving beyond rankings.

While the existing literature establishes the technical validity of K-means (Matlis, et al., 2024; Kucukdeniz, et al., 2025) and the strategic necessity of moving toward "fit-based" evaluation (Poole, et al., 2018), there remains a significant gap in the analysis of highly concentrated metropolitan markets. London represents one of the world's most dense and diverse higher education landscapes, yet most studies remain national or global in scale. This project addresses that gap by applying the methodologies validated by previous research using K-Means for analysis to the specific tension between prestige and performance within 22 institutions based in London.

The 22 HEIs consist of non-specialized, undergraduate-focused, traditional universities physically located in London. Satellite campuses of other universities, specialized colleges, alternative tertiary institutions and postgraduate universities are not included in this project.

6.0 REQUIREMENTS SPECIFICATION

6.1 Methodology

The requirements for this project were identified using a user-centric needs analysis approach. As the primary users are likely to be prospective university applicants, requirements were captured by identifying the informational blind spots in traditional league tables; specifically, the need for a non-linear comparison of prestige versus student satisfaction.

Following the principles outlined by Sommerville (2016), the requirements were categorized into functional requirements, which define what the system must do, and non-functional requirements, which define the quality and constraints of the system. This ensures the technical artefact is not only mathematically sound but also practically useful for decision-making.

6.2 Functional Requirements

The functional requirements of the system are outlined below:

1. The system must ingest and clean multi-source datasets for the 22 selected London universities.
2. It must implement a K-Means Clustering algorithm to group institutions into distinct archetypes.
3. The system must calculate a composite Course Value Index based on weighted satisfaction, prestige and employment metrics.
4. The dashboard must provide a subject-level filter allowing users to recalculate rankings based on specific fields of study.
5. The artefact must visualize statistical consistency using Box-and-Whisker plots to validate cluster homogeneity.

6.3 Non-Functional Requirements

The following are the non-functional requirements for this project:

1. Usability: The dashboard must follow Shneiderman's Eight Golden Rules of Interface Design (GeeksforGeeks, 2023), specifically providing informative feedback via interactive tooltips.
2. Accuracy: The clustering model must be validated via the Elbow Method or Silhouette Analysis to ensure the optimal number of clusters is selected.

3. Performance: Filter changes on the Tableau dashboard must update all ten visualizations in under 2 seconds.

6.4 Legal, Ethical, Environmental and Professional Constraints

- ### Legal and Ethical Issues

a. Data Provenance: The project utilizes public-sector data from HESA, Discover Uni and the Office for Students (OfS). While this is open data, the project adheres to the Higher Education Statistics Agency (HESA) Standard Selection guidelines, ensuring no individual student data is deanonymized (HESA, 2026).

b. Algorithmic Bias: A significant ethical constraint is the potential for "prestige bias." To mitigate this, the clustering algorithm was designed to weight student-reported satisfaction (NSS) and employment outcomes equally with institutional entry standards, preventing the model from simply replicating traditional reputation-based hierarchies.

- ### Professional Issues

This work adheres to the British Computer Society (BCS) Code of Conduct, specifically regarding section 2, "Professional Competence and Integrity" (BCS, 2022). This is achieved by documenting all data cleaning steps and ensuring the statistical visualizations transparently report on missing or suppressed data rather than hiding it.

- ### Environmental Issues

While the computational requirements for clustering 22 institutions are relatively low, the project minimizes its carbon footprint by utilizing local optimized compute environments (Python/Tableau Desktop) rather than high-energy cloud-server instances for the iterative development phase.

7.0 TECHNICAL DESIGN

7.1 System Architecture

The project utilizes a decoupled architecture, separating the data processing layer from the visualization layer. This modular approach ensures that the clustering logic can be updated or retrained without requiring a redesign of the user interface.

- Ingestion Layer: Raw data acquisition from HESA (Employment/Enrollment), Discover Uni, QS and THE.
- Processing Layer (Python/Pandas): Data cleaning, handling of suppressed values, and feature engineering (calculating the Course Value Index).
- Algorithmic Layer (Scikit-Learn): Implementation of K-Means clustering, including feature scaling (Min-Max Normalization) to ensure all metrics carry equal weight.
- Presentation Layer (Tableau): Integration of the clustered CSV into an interactive dashboard environment.

7.2 Algorithmic Design: K-Means Implementation

To ensure mathematical rigour, the design of the clustering component followed a three-stage sequence:

- Feature Selection: Identifying the core variables (prestige, satisfaction and employment outcomes) that define institutional value.
- Normalization: Since variables like exist on different scales, the design includes a Min-Max Scaling step to transform all data to a 0 to 1 range, preventing large-magnitude variables from dominating the centroids.
- Optimal K Selection: The design utilized the Elbow Method to plot the Within-Cluster Sum of Squares (WCSS), identifying the elbow point where adding more clusters yielded diminishing returns in variance reduction.

7.3 The Interactive Dashboard

The final artefact consists of ten distinct visualizations and one dashboard, designed for easy viewing and intuitive usage.

- Global Interactivity: The design utilizes a global filter on the Course/Subject, HE provider and Cluster Category attributes. This allows the user to re-calculate an entire visualization for a specific subject (e.g., Computing) with a single click.

- Visual Encoding: Following Shneiderman's Eight Golden Rules (GeeksforGeeks, 2023), colour is used consistently across all ten charts (e.g., orange always represents Elite Academic Tier cluster). This reduces the cognitive load on the user as they navigate different perspectives of the data.

7.4 Data Structure Design

The output of the design phase is a Unified Master Schema. Unlike the fragmented source files, the design produced a flattened CSV structure where every row represents a unique Higher Education Provider + Subject combination, enriched with a Cluster Category tag. This schema design allows Tableau to perform complex aggregations at runtime without performance degradation.

8.0 TECHNICAL DEVELOPMENT

8.1 Development Environment

The project was developed using a hybrid stack designed for high-performance data manipulation and interactive visual storytelling:

- **Data Processing**: Python 3.11 using the Pandas and NumPy libraries for ETL (Extract, Transform, Load) processes and Scikit-Learn for the implementation of the K-Means algorithm.
- **Integrated Development Environment (IDE)**: Jupyter Notebooks was utilized for iterative script testing and intermediate data profiling.
- **Visualization Engine**: Tableau Public was selected over Python-based libraries (like Matplotlib) to fulfill the requirement for a consumer-grade, interactive end-user interface.
- **Hardware**: An Intel Core i9 Ultra system with 32GB RAM was used to ensure zero latency when processing the 200+ attribute high-dimensional dataset.

8.2 Project Plan and Execution

This project followed a simple Kanban methodology due to its simplicity and straightforward approach. The project was divided into smaller sub-tasks, which are outlined below:

1. **Data Gathering**: The data for this project was gathered manually from HESA, Discover Uni, QS and THE. It was done manually due to how easily accessible the data was, leaving no real need for scripting a web scraper.
2. **Data Cleaning and Pre-Processing**: The gathered data was loaded, cleaned and combined into one large dataset in Python using Pandas and NumPy.
3. **Data Analysis**: A custom Prestige Score and Custom Value Index were calculated for each university.
    a. ***Prestige Score***

    The raw ranking data (e.g., QS Rank) is ordinal, meaning a lower number is better. To include this in a "higher is better" index, the values were normalized and inverted.

    The formula:
    Prestige_Score = (1- (Specific University Rank/ Max Rank in Dataset)) * 100

Normalization: By dividing the university's rank by the maximum rank (the lowest-ranked institution in the London dataset), the value is scaled to a decimal between 0 and 1.

Inversion: Subtracting that decimal from 1 ensures the 1st-ranked university receives a score near 100, while the lowest-ranked institution receives a score near 0.

Handling Unranked Schools: If a university was not present in the rankings, it was assigned the maximum rank (0 points) to maintain a conservative estimate.

b. Course Value Index

This index is a composite metric that evaluates a specific course based on three distinct pillars: graduate outcomes, student satisfaction, and institutional reputation.

The formula:

Course_Value_Index = (Outcome * 0.4) + (Satisfaction * 0.3) + (Reputation * 0.3)

The pillars are weighted as follows:

Outcome (40% Weight): Represented by the WORK column from Discover Uni (percentage of graduates in work or further study). This was prioritized as the most significant factor for prospective applicants.

Satisfaction (30% Weight): Represented by Q27 from the NSS (National Student Survey), which measures overall satisfaction. This ensures the score reflects the actual student experience.

Reputation (30% Weight): Represented by the Prestige_Score derived from global rankings. This accounts for the institutional brand value.

This approach uses a linear combination. By combining these factors, the index identifies high-value courses that might not be at the most famous universities but have exceptional employment rates or satisfaction scores. Using Scikit-Learn, K-Means clustering was then applied to the dataset to sort the HEIs into four distinct groups.

4. **Data Visualization**: The clustered dataset was then loaded into Tableau, where all the visualizations where carried out and an interactive dashboard was built.

8.3 Data Analysis and Pipeline Design Decisions

A critical design decision was to move from a normalized database structure to a denormalized flat file for the final output.

- **The Problem**: HESA and NSS data arrive as separate relational tables linked by provider codes. Performing complex joins inside Tableau for every subject-level filter change created a performance bottleneck (non-functional requirement 3).
- **The Resolution**: The Python script was designed to perform all heavy joins and feature engineering (such as the Course Value Index calculation) upstream. This resulted in a single FINAL_CLUSTERED_LONDON_COURSES.csv, optimized for Tableau's hyper-engine.

8.4 Handling Uncertainties and Requirements Resolution

A significant technical challenge during the development phase was the overwhelming volume of the raw national datasets. The initial HESA data was pre-filtered to include only London-based institutions, while the Discover Uni dataset included hundreds of institutions across the UK, resulting in a high-dimensional matrix that threatened the computational efficiency and interpretability of the K-Means model.

- **The Problem:** Processing thousands of rows from non-comparable regional institutions (e.g., rural vs. metropolitan) would make it difficult for the algorithm to define clear centroids, as the unique economic variables of the London job market would be diluted by national averages.

- **The Resolution:** To satisfy the requirement for a granular, student-centric analysis, the strategic decision was made to filter the dataset to focus exclusively on the 22

selected London universities. This reduced the data volume to a "high-signal" subset, ensuring the clustering logic was applied only to a comparable peer group.

By narrowing the scope two key issues were solved:

- **Computational Manageability**: The resulting master file became significantly smaller and lighter, allowing for faster iterations of the K-Means algorithm and smoother performance in the Tableau environment.

- **Algorithmic Precision**: The centroids would now reflect the specific value of only London-based institutions, providing prospective applicants with a granular, like-for-like comparison that a national-scale model would not be able to achieve.

This pivot from a broad but noisy to a smaller but deeper data structure ensured the final artefact delivered the specific regional insights promised in the project specification.

8.5 Discussion of Problems and Risk Mitigation

Consistent with the Risk Register (Appendix B), two primary technical risks materialized:

1. **Entity Resolution Due To Inconsistent Data**: Different datasets used varying names for the same HE providers (e.g., "KCL" vs. "King's College London"). This was mitigated by using a government-provided institution number called UKPRN to identify universities instead of using the institution names for clustering. After merging the data and performing clustering, the institution names were then re-matched and merged from one specific dataset to the master file using the UKPRN.

2. **The Data Ghost Problem**: Certain institutions and courses had rows where small student numbers were hidden for privacy or lack of data. Using such data could result in skewed groups, affecting the quality of the clusters. Rather than deleting these rows and losing institutional context, a designated "Developing/Entry Level" cluster was created(denoted by blue in the visualizations). This isolated the noise and ensured the integrity of the remaining active clusters.

8.6 Progressive Refinement: Implementing K-Means

To determine the optimal number of clusters, the Elbow Method was employed. By plotting the Within-Cluster Sum of Squares (WCSS) against a range of k values (1-10), a clear inflection point was observed at k=4. This indicated that four clusters provided the ideal balance between model complexity and group homogeneity, effectively isolating the distinct institutional archetypes found in the London HE sector.

In Tableau, a subject-level filter was incorporated. This refinement allowed the model to be dynamic, proving that a university's cluster can shift depending on the academic discipline, which was a key finding of the project.

9.0 PROJECT EVALUATION

9.1 Evaluation Strategy

The evaluation of this project was conducted using a dual-track approach to ensure both mathematical rigor and interface effectiveness. Each evaluation track is as follows:

1. **Algorithmic Evaluation**: To ensure the K-Means model correctly identified meaningful patterns rather than random noise.
2. **Usability Evaluation**: To judge the Tableau interface's effectiveness in helping prospective applicants navigate complex data, following heuristic evaluation principles.

9.2 Algorithmic Evaluation (Summative Model Testing)

The clustering logic was evaluated using internal validation metrics to confirm the stability of the four identified clusters:

- **The Elbow Method**: As described in the development chapter, the Within-Cluster Sum of Squares (WCSS) was calculated for k=1 through k=10. The evaluation focused on identifying the point of diminishing returns to justify the final model.
- **Centroid Proximity Analysis**: Each cluster's "DNA" was evaluated by measuring the distance of individual data points (universities) from their respective centroids. This was visually verified using the Cluster Consistency Box Plots in Tableau (Cluster Consistency Box Plot), which judged whether the clusters maintained high internal homogeneity (tight boxes) and clear external separation (no overlap).

9.3 Usability Evaluation

To evaluate the Tableau dashboard as a decision-making tool, a heuristic evaluation was performed based on Shneiderman's Eight Golden Rules of Interface Design (GeeksforGeeks, 2023). The following components were judged:

- **Consistency (Rule 1):** The dashboard was evaluated for visual consistency, ensuring that color-coding (e.g., Orange for "Elite All-Rounders") remained identical across all ten visualizations to reduce cognitive load.
- **Informative Feedback (Rule 3):** Evaluation focused on the "hover-over" tooltips. The system was tested to ensure that when a user selects a university dot, the dashboard provides immediate, granular data (NSS scores, exact employment percentages) to support a deeper understanding of the data.

- **User Control and Freedom (Rule 5):** The Global Subject Filter (NSSSBJ) was evaluated through scenario-based testing. For example, can a user looking for a computing degree at a mid-prestige university quickly isolate relevant 'Value Specialists'? The evaluation judged the system's responsiveness and the accuracy of the re-sorting logic.

9.4 Requirements Traceability Evaluation

A final summative evaluation was conducted by mapping the finished artifact back to the original Requirements Specification in section 6.

- **Functional Testing**: Each visualization was cross-referenced against the functional requirements (refer to section 6) to ensure all intended features (such as the Prestige vs. Entry Standards scatter plot) were present and operational.

- **Performance Testing (Non-Functional Requirement Evaluation):** The dashboard's response time was evaluated against the 2-second threshold (third non-functional requirement specified in section 6). This involved stress-testing the subject filter with "All" subjects selected to ensure the data engine could manage the maximum aggregation load without latency.

10.0 RESULTS OF EVALUATION

10.1 Algorithmic Evaluation Results

The following are the results of the algorithmic evaluations highlighted in the earlier section.

A. The Elbow Curve and WCSS

The K-Means algorithm was executed for $k=1$ through $k=10$. The resulting Within-Cluster Sum of Squares (WCSS), being the "Inertia" of the model, is recorded in the table below.

Table 1: WCSS Values for Cluster Validation

Number of Clusters (k)	**WCSS (Inertia)**
k=1	310,124.00
k=2	195,803.73
k=3	133,478.93
k=4 (Optimal)	97,618.41
k=5	74,570.05
k=6	62,118.58
k=7	54,493.52
k=8	47,300.30
k=9	42,951.62
k=10	38,869.82

The elbow plot generated from these values showed a distinct inflection point at k=4 (Figure 1). While the WCSS continues to decrease as k increases, the marginal gain in cluster tightness drops significantly after k=4. Thus, this is the optimal balance between model complexity and data interpretability.

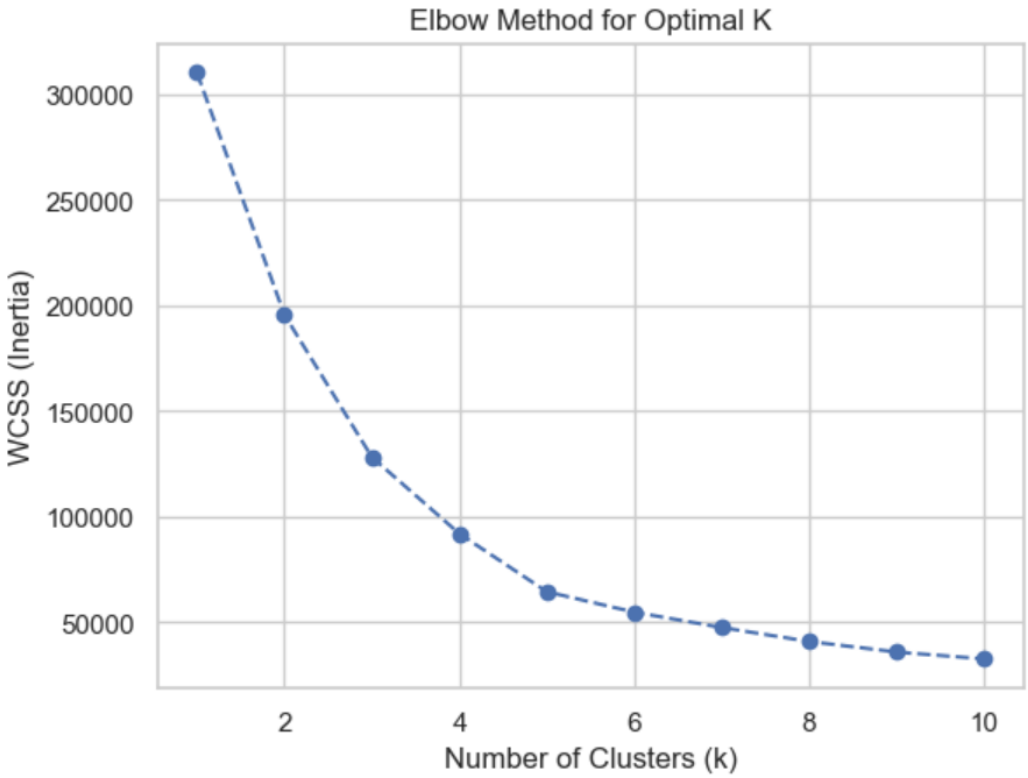


Figure 1.: Elbow Plot for Optimal K

B. Cluster Consistency (Box Plot Results)

The internal homogeneity of the clusters was verified using box-and-whisker plots for the Course Value Index (Figure 2). The results showed:

- Zero Inter-Quartile Overlap: The 25th to 75th percentile boxes for the "Elite Academic Tier" (Orange), "Specialist and Niche" (Green), and "High-Value Career Hub" (Red) were entirely distinct on the Y-axis.

- Centroid Variance: The Elite Academic Tier cluster showed the narrowest interquartile range (IQR), while the High-Value Career Hub cluster showed a wider distribution with specific high-performing outliers.

- Data Isolation: The Developing/Entry Level (Blue) cluster was successfully isolated at the base of the scale, being institutions with suppressed or limited data points.

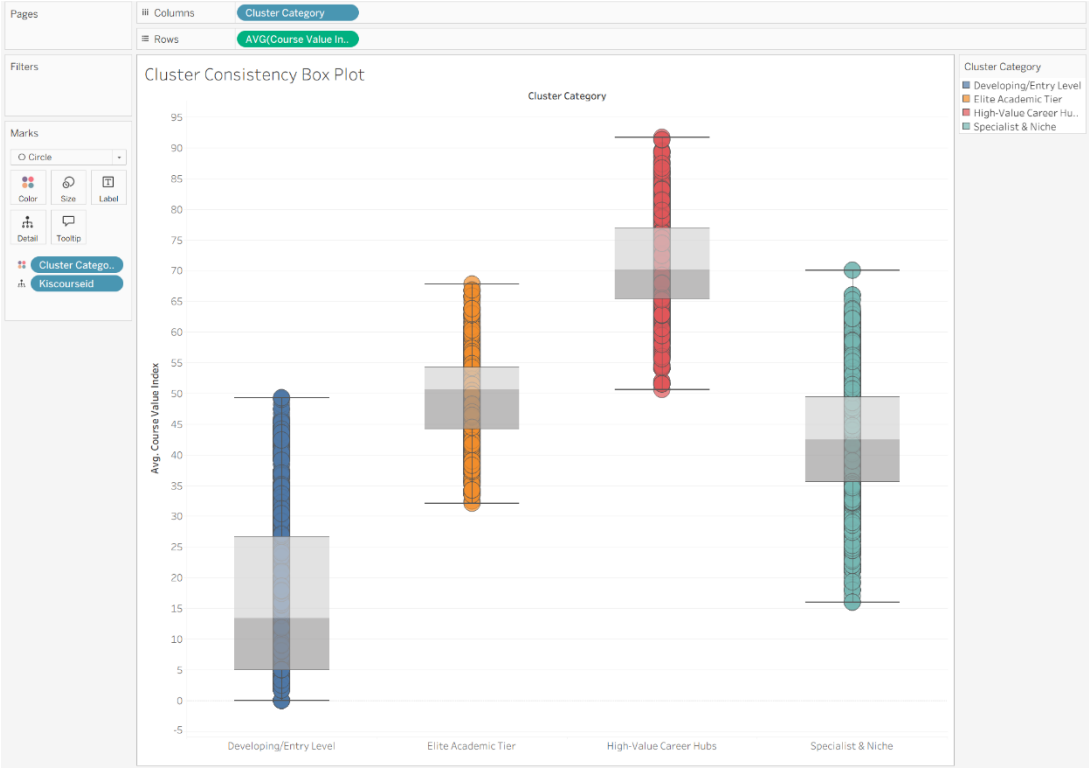


Figure 2: Cluster Consistency Box Plot

10.2 Usability Evaluation Results

The following are the results of the usability evaluation:

- **Visual Consistency**: 100% of the visualizations across all dashboard tabs utilized the same colour-to-cluster mapping and font hierarchy.

- **Tooltip Richness**: Testing confirmed that hovering over any data point triggered a dynamic tooltip containing data attributes relevant to the specific visualization (e.g., Provider Name, Course Value Index, Employment %, Prestige Rank, etc.).

- **State Persistence**: The Global Subject Filter successfully updated all dashboard and workbook visualizations simultaneously without resetting the zoom levels of the scatter plots.

10.3 Requirements Traceability Evaluation Results

The results from the requirements' traceability evaluation are outlined in the table below:

Table 2: Requirements Traceability Evaluation Table

Requirement	**Description**	**Status**
Functional Requirement 1	Multi-source Data Ingestion	Pass
Functional Requirement 2	K-Means Algorithm Implementation	Pass
Functional Requirement 3	Course Value Index Calculation	Pass
Functional Requirement 4	Global Subject-Level Filter	Pass
Functional Requirement 5	Statistical Consistency Visualization	Pass
Non-Functional Requirement 1	Shneiderman Heuristic Compliance	Pass
Non-Functional Requirement 2	Algorithmic Accuracy (Elbow Method)	Pass
Non-Functional Requirement 3	System Latency (< 2s)	Pass

11.0 DISCUSSION

11.1 The Anatomy of the London Market: Decoding the Archetypes

The primary finding of this project is that "institutional value" in London is not a linear hierarchy, but a multi-dimensional landscape. The K-Means clustering successfully moved beyond the reductionist approach of league tables by identifying four distinct archetypes, namely:

- **The "Elite Academic Tier" (Orange Cluster)**: These institutions represent the successful alignment of prestige and decent career outcomes. The cluster exhibits a high degree of market efficiency, having high entry standards (Average Estimated Entry Tariff) and high employment (Work) scores, as seen in Figures 4 and 5. However, the analysis reveals a satisfaction ceiling, as these institutions often score lower on NSS metrics than their less-prestigious counterparts, suggesting that their academic rigour may diminish the perceived student experience (Figure 3).

- **The "High-Value Career Hubs" (Red Cluster)**: This is perhaps the project's most significant discovery. These institutions act as local powerhouses, with prestige scores rivalling that of the Elite Academic Tier while having similar employment outcomes and high student satisfaction scores (Figures 3 and 4). This finding validates the arguments of Poole et al. (2018) that the university ranking alone is not enough to find the "best-fit" institution for prospective students.

- **The "Specialist & Niche" (Green Cluster)**: These institutions offer high value for certain courses but lack the general prestige of the first two groups, as seen in Figure 3. With typically lower entry standards (Figure 5), these institutions possess surprisingly decent employment outcomes, which can make them worth considering for certain niche courses (Figure 4). In traditional league tables, these would typically rank lower, but this analysis shows that these universities can compete with higher-ranked ones in the subjects that they specialize in.

- **The "Developing/Entry Level" (Blue Cluster)**: This cluster represents institutions or specific courses that have limited data (Figure 3). A lot of these are smaller, specialized HEIs or new courses from established universities.

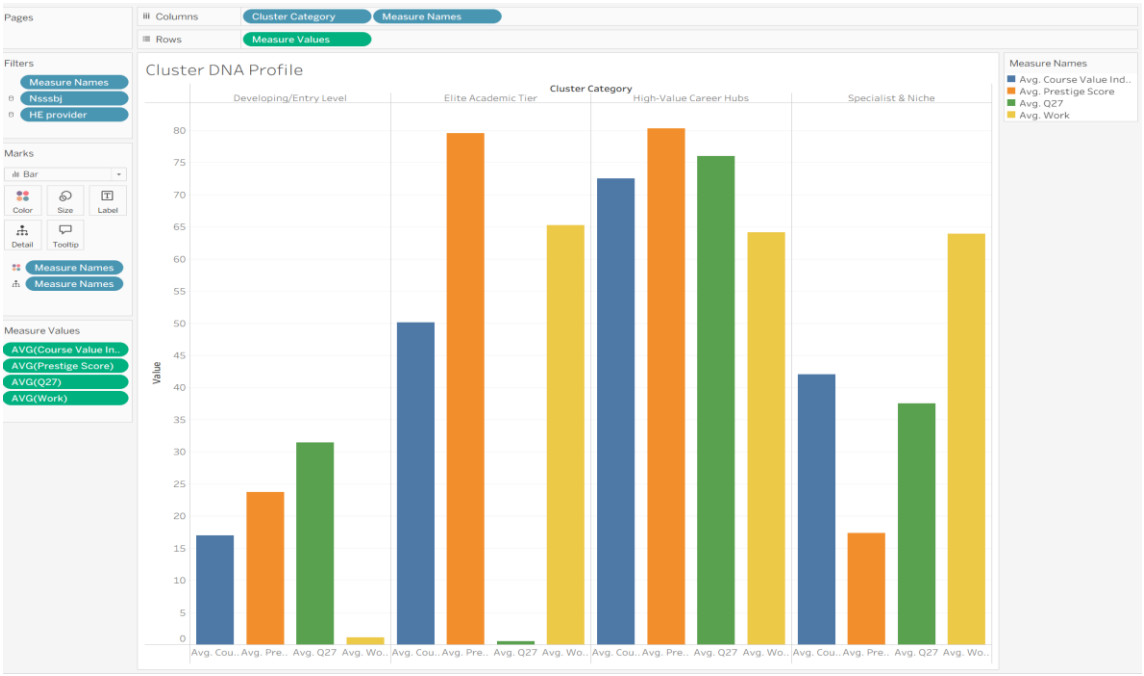


Figure 3: Cluster DNA Profile Bar Chart Highlighting Each Cluster



Figure 4: Hidden Gems Scatter Plot Highlighting Prestige vs. Employment Outcomes



Figure 5: Scatter Plot Highlighting Prestige vs. Entry Standards

11.2 The Prestige-Tariff Paradox and Gatekeeping

The visualization comparing Entry Standards (Tariff) to Prestige Score (Figure 5) reveals a critical industry insight. While a general linear correlation exists, the vertical spread in the high-prestige range indicates significant “gatekeeping” variance. Certain elite institutions maintain exceptionally high tariff bars that exceed their relative career-value output. This suggests that for some Elite Academic Tier institutions, the entry requirement acts more as a brand-protection mechanism than a prerequisite for professional success. Conversely, the “accessible prestige” outliers identified in the bottom-right of the plot offer a higher return-on-entry for students who may not meet the maximum tariff scores but seek high-brand degrees.

11.3 Subject-Level Sensitivity and Institutional Quality

The results from the Global Subject Filter prove that institutional quality is highly volatile. A university belonging to the High-Value Career Hub (Red) cluster for Business may shift into the Developing/Entry Level or Specialist & Niche category for Creative Arts. This variability critiques the universal ranking model used by major league tables, proving that an applicant’s decision must be subject-first, rather than institution-first.

11.4 Critical Analysis of Graduate Outcomes and Diversity

The analysis of the percentage of International Students vs. Employment Outcomes in Figure 6 identified a strong positive correlation within the Specialist & Niche (Green) cluster. This might suggest that specialized programs gain more professional value as they become more globally diverse.

However, this relationship was negative in the Elite Academic Tier (Orange) and High-Value Career Hubs (Red) clusters, where both of these high-performing clusters show a downward trend. This could be due to various factors, such as international students returning to home markets (which may be tracked differently) or specific visa-related employment hurdles.

The implication of this is that while global status and diversity are often associated with elite brands, it does not necessarily increase employment outcomes. However, the Elite Academic Tier (Orange) and High-Value Career Hubs (Red) clusters still maintain high work outcomes overall, thus making them safe choices for international students.



Figure 6: Scatter Plot for Percentage of International Students vs. Employment Outcomes

11.5 The Impact of Prestige on Student Satisfaction

Based on the analysis of Prestige vs. Satisfaction in Figure 7, it is apparent that the High-Value Career Hub (Red) cluster is the gold standard for student satisfaction, as it has the

highest satisfaction scores among any cluster. However, it cannot be implied that prestige is the sole reason for such high student satisfaction, as the Elite Academic Tier (Orange) cluster has the lowest student satisfaction rates despite having strong prestige.

The Elite Academic Tier (Orange) cluster is the most interesting and perhaps concerning area, as it could point to "academic pressure cooker" environments where the brand name is strong, but the student experience is lacking.

Unsurprisingly, the Specialist & Niche (Green) cluster has a wide range of satisfaction scores, all the way from low to high, which is in line with the cluster's course-dependent nature.

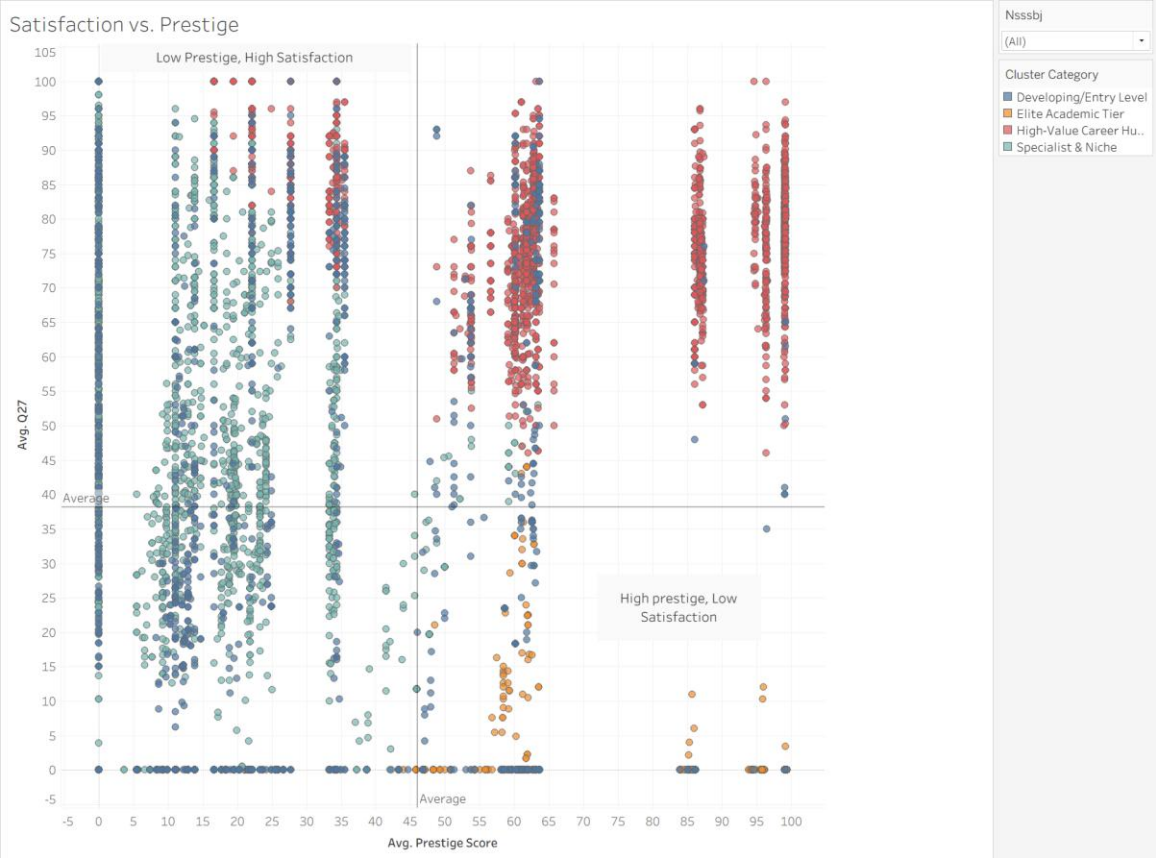


Figure 7: Scatter Plot of Prestige vs. Student Satisfaction

11.6 Spatial Dynamics: The Centrality of International Appeal

The map visualization of the International Student Ratio in the 22 selected universities reveals a clear geographic-demographic correlation (Figure 8). By mapping international student populations across the London boroughs, a "central hub" phenomenon that transcends individual university clusters has been identified.

- **The Scale-Internationalization Loop**: There is a visible impact of total institutional population on international recruitment. The largest bubbles on the map correspond to institutions with the highest overall enrollment. This suggests a gravity effect, where larger institutions leverage their scale to build global brand recognition, which in turn attracts a higher volume of non-UK students.

- **Centrality as a Prestige Proxy:** The visualization also shows a heavy concentration of international students in the Central London boroughs. While the Elite Academic Tier (Orange) institutions typically dominate these central locations, the map reveals that even smaller providers in the city centre often maintain a higher international-to-domestic ratio than larger providers in the outer zones. This suggests that for international applicants, London is often viewed as a singular, central brand, benefiting any institution located within the primary metropolitan core.

- **Critical Implications for Social Mobility**: Universities located further from the central core typically have lower international numbers but as shown in previous visualizations, maintain high employment outcomes for domestic students. This implies a strategic divide where central institutions act as global hubs for international talent, while outer-zone institutions act as social mobility engines for the local London population.

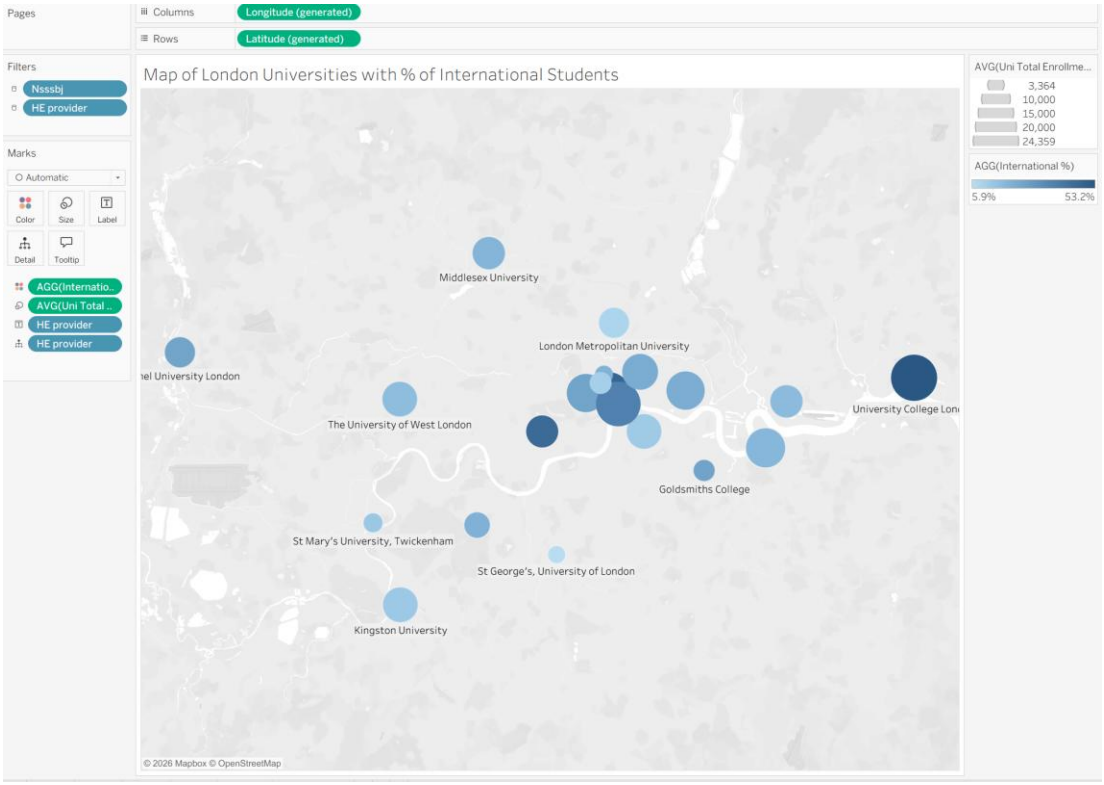


Figure 8: Map of London Universities Showing Enrollment Size and International Population Percentages

11.7 Ranking Dissonance in Traditional League Tables

The aim of this project is to provide a tool to complement traditional university ranking tables. In light of this, a visualization comparing each of the universities' QS rank and THE rank was created. However, seeing the HEIs rankings on the different tables juxtaposed highlighted the earlier-mentioned issues of the linear ranking model. These are outlined below:

- **The Consensus of the Elite**: The visualization shows a tight correlation among the top five institutions appearing on both rankings. For universities like Imperial College London, University College London (UCL), London School of Economics and Political Science (LSE)and King's College London, both ranking bodies show relative agreement. This confirms a "brand consensus" in the global market, where high-prestige institutions maintain their status regardless of the ranking criteria used.

- **The Divergence Outliers (Methodological Bias)**: Significant ranking dissonance is visible among mid-tier institutions. In several cases, an institution may rank significantly higher in QS than in THE (or vice versa). Some institutions are completely

missing in one ranking while being present in the other. This divergence highlights the differing weights these bodies place on metrics such as Employer Reputation (favoured by QS) versus Research Citations and Teaching Environment (favoured by THE), as well as the preference some institutions may have toward being included in one ranking because it possibly suits their strengths, but not being included in the other table.

- **The Case for Clustering**: This discrepancy validates the primary thesis of this research; linear rankings are inherently biased by their own weighting systems. By using both rankings as just one component of a multi-dimensional K-Means model alongside student-reported satisfaction and actual career outcomes, this project provides a more stable and unbiased view of institutional value than any single global league table can offer.

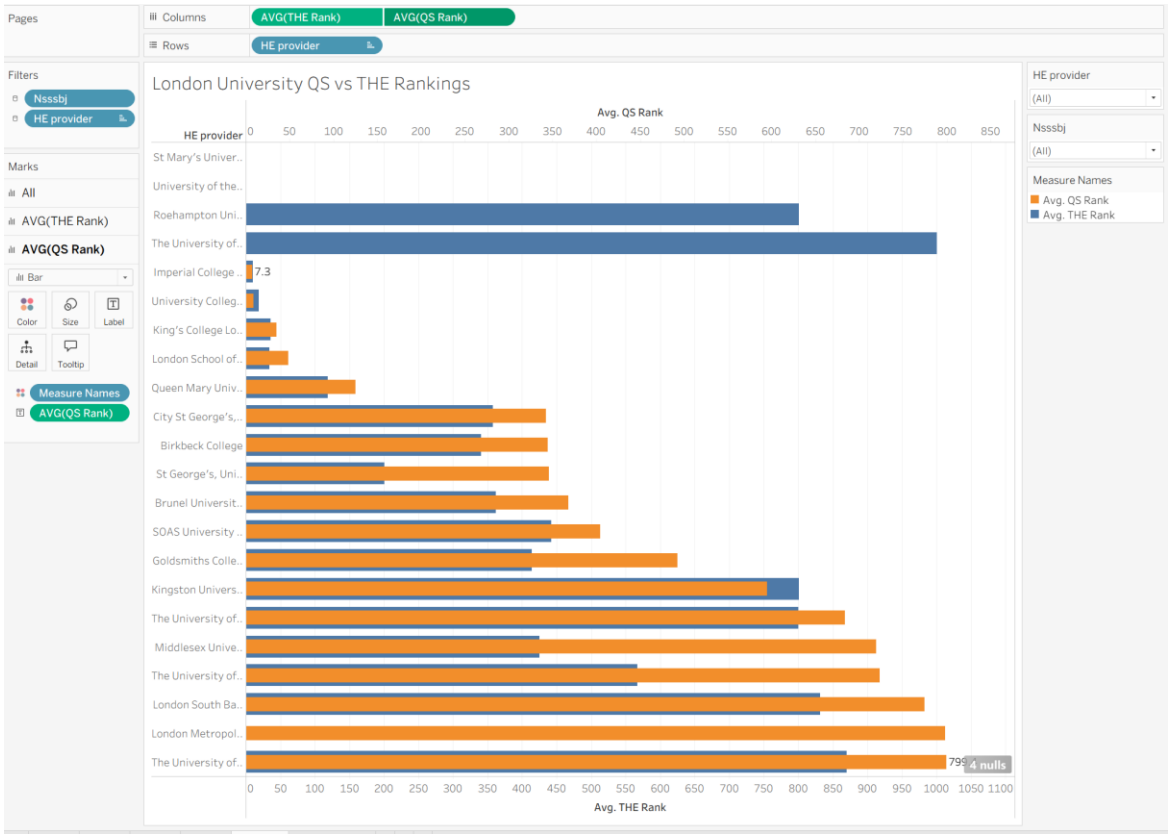


Figure 9: QS Vs. THE Ranking for Each of the 22 Universities

11.8 Dynamic Dashboard for Customizable Comparisons

The final technical achievement of this project is the integration of the most relevant findings into a single, synchronized dashboard environment. Unlike traditional league tables that offer static, non-negotiable rankings, this interactive artefact allows the user to perform real-time, multi-dimensional comparisons.

- **Global State Synchronization**: As evidenced in the final dashboard layout, the implementation of global filters (specifically HE Provider, Cluster Category and Subject (NSSSBJ)) ensures that every visualization updates in unison. This technical choice maintains contextual integrity; such that when a user selects a specific discipline like "Computing," the scatter plots, maps, and leaderboards re-align instantly to show the specific market reality for that field.

- **Prioritization of Best-Fit**: This dashboard directly addresses the core limitation of standard rankings, which is rigid and hierarchical by nature. By allowing users to toggle between different subjects or filter by specific academic clusters, the dashboard proves that an institution's value is dynamic. A university that appears as a Specialist & Niche institution in a general view may reveal itself as an Elite Academic Tier when filtered for a specific high-growth subject.

- **Fair Assessment for HEIs**: The dashboard's informative feedback loop (via tooltips and synchronized filtering) moves the applicant from a passive consumer of prestige to an active researcher. It allows for the discovery of those institutions that may be geographically distant or lower-ranked globally but show superior performance metrics when the filters are tuned to the user's specific needs. The Prestige Vs. Value visualization helps identify institutions that may be hidden gems providing more value than traditional rankings account for. If a hierarchical ranking is still needed, the University Ranking by Course Value Index visualization allows for a dynamic list of HEIs based on the custom Course Value Index, which can be adjusted using any of the filters for full granularity.

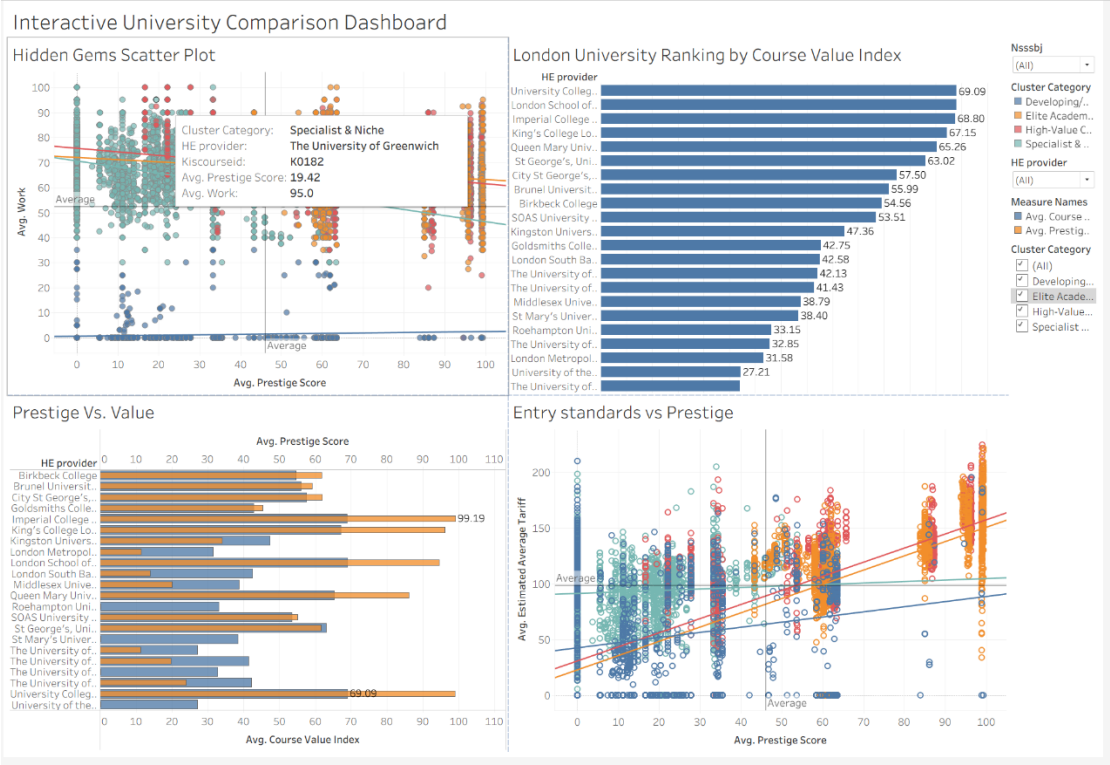


Figure 10: Dynamic University Comparison Dashboard

11.9 Methodological Limitations

While the project met its technical requirements, several limitations must be critically acknowledged:

- **Data Sparsity and the Developing/Entry Level (Blue) Cluster**: The existence of the blue cluster is a professional concern. It represents a transparency gap in the London market, where smaller or highly specialized providers do not meet the minimum HESA reporting thresholds. This limits the ability of the algorithm to provide a fair evaluation for these specific schools.

- **Cross-Sectional vs. Longitudinal**: The data represents a snapshot (2020-2024). It does not account for the long-term career trajectory of graduates (5-10 years post-degree), which is a critical component of true institutional value.

12.0 CONCLUSION

12.1 Achievement of Objectives

The project was evaluated against the initial objectives set out in the requirements specification. The primary goal of developing a clustering-based analytical tool for London HEIs was successfully achieved.

- **Interactive Metric Visualization (Achieved):** The technical artefact met all performance requirements, delivering a dashboard that allows for real-time subject-level filtering, bridging the gap between raw data and applicant decision-making.
- **Subject-Level Granularity (Achieved):** The dashboard allows for subject-level filtering across all visualizations.
- **Comparative Analysis Engine (Achieved):** The artefact allows comparison of any given number of universities by their Course Value Index, Prestige Score and employment outcomes.
- **The Course Value Index (Achieved)**: This was calculated for each course at every institution and used for the clustering analysis.
- **Cluster Discovery (Achieved)**: Four distinct clusters were identified and analyzed, each with their own strengths and weaknesses.
- **Demographic Mapping (Achieved)**: analyzed and visualized international student ratios in Tableau.
- **Operational Metrics (Achieved)**: visualized institutional scale and spatial distribution across London boroughs.

- **Implementation of a Data-Driven Clustering Model (Achieved)**: The project successfully integrated HESA and NSS data to produce a stable K-Means model. The mathematical validation via the Elbow Method (k=4) ensured that the resulting Elite Academic Tier cluster and High-Value Career Hub cluster archetypes were statistically significant rather than arbitrary.

12.2 Reflection on Limitations and Failed Objectives

In the interest of professional integrity and self-critical reflection, it must be acknowledged that certain advanced objectives were only partially met or strategically abandoned.

- **The Degree Classification Constraint (Failed/Modified):** A secondary objective was to correlate clustering with total academic attainment, including lower-second and third-class honours. As discussed in the development chapter, this objective was not achieved due to the high volume of suppressed or sparse data in the HESA source files. While the pivot to a Course Value Index maintained the project's integrity, the omission of granular academic attainment limits the model's ability to judge academic rigour vs. grade inflation.
- **Data Completeness and the Developing/Entry Level Cluster (Limitation):** The initial aim was to provide a comprehensive map of all London providers. However, the requirement to maintain signal-to-noise led to the exclusion of smaller providers or their isolation into the data-limited Developing/Entry Level (Blue) cluster. This reflects a persistent challenge in the HE sector data transparency is often a privilege of larger, well-funded institutions.

12.3 Personal and Professional Reflection

Completing this project has been a transformative experience in applied data science. From a technical perspective, I have learned the importance of data skepticism; the realization that just because a column exists in a dataset does not mean it is robust enough for machine learning. The decision to filter the scope to 22 institutions was a difficult but necessary exercise in engineering pragmatism, ensuring the final product was usable and accurate.

As a student, this project has deepened my understanding of the value gap. Seeing the prestige dissonance between QS and THE rankings confirmed my suspicion that the brand of a university is often a social construct, while value is a data-driven reality.

12.4 Future Work

If this research were to be extended, two areas would be prioritized:

1. **Longitudinal Integration:** Incorporating five-year post-graduation salary data to see if any of the clusters, particularly the High-Value Career Hub (Red) cluster, maintains its performance over a decade.
2. **Predictive Modelling:** Moving from unsupervised clustering to a supervised model that could predict an institution's future satisfaction scores based on current investment and enrollment trends.

12.5 Final Summary

This project proves that institutional value is not a ranking, but a relationship between prestige, career outcome, and personal satisfaction. By moving beyond league tables, this

technical artefact provides a roadmap for a more honest and student-centric evaluation of higher education in the world's most competitive academic market.

13.0 BIBLIOGRAPHY

Al-Hagree, S. et al., 2023. Universities the best performers: A cluster analysis of Yemeni universities rankings.. *2023 3rd International Conference on Emerging Smart Technologies and Applications (eSmarTA)*, October, pp. 1-9.

BCS, 2022. *CODE OF CONDUCT FOR BCS MEMBERS*. [Online] Available at: https://www.bcs.org/media/2211/bcs-code-of-conduct.pdf [Accessed 21 April 2026].

GeeksforGeeks, 2023. *Ben Shneiderman eight golden rules of interface design (Human Computer Interaction)*. [Online] Available at: https://www.geeksforgeeks.org/software-engineering/ben-shneiderman-eight-golden-rules-of-interface-design-human-computer-interaction/ [Accessed 12 April 2026].

HESA, 2026. *Open data and official statistics*. [Online] Available at: https://www.hesa.ac.uk/data-and-analysis [Accessed 1 March 2026].

Kucukdeniz, T., Altuntas, M., Akarsu, C. & Ince, E., 2025. Beyond Rankings: A K-means Approach to Evaluating Research Universities in Emerging Higher Education Systems. *International Journal of Applied Mathematics, Computational Science and Systems Engineering,* Volume 7, pp. 51-63.

Matlis, G., Dimokas, N. & Karvelis, P., 2024. Unveiling university groupings: A clustering analysis for academic rankings. *Data,* 9(5), p. 67.

Poole, S., Levin, M. & Elam, K., 2018. Getting out of the rankings game: a better way to evaluate higher education institutions for best fit. *Journal of Marketing for Higher Education,* 28(1), pp. 12-31.

Sommerville, I., 2016. *Software Engineering*. 10th ed. Essex: Pearson.

14.0 APPENDICES

APPENDIX A: SOURCE CODE

Link to Tableau Dashboard: https://public.tableau.com/views/BeyondLeagueTables-UsingK-MeansToEvaluate22LondonUniversities/Dashboard1?:language=en-GB&publish=yes&:sid=&:redirect=auth&:display_count=n&:origin=viz_share_link

Link to Tableau Workbook: https://drive.google.com/file/d/19N29TeuYBC1MDmA4MF1BKp69Sg9Llxld/view?usp=sharing

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:


import pandas as pd
import glob
import re
import os

def clean_hesa_enrollments(file_pattern):
    """
    Cleans and merges multiple HESA enrollment CSV files.
    """
    all_data = []

    # Use glob to find all files matching the pattern
    files = glob.glob(file_pattern)

    for file_path in files:
        print(f"Processing: {file_path}")

        # Extract Academic Year from the metadata
        academic_year = "Unknown"
        with open(file_path, 'r', encoding='utf-8-sig') as f:
            lines = f.readlines()

            for line in lines[:20]:
                if "Academic year" in line:
                    match = re.search(r'\d{4}/\d{2}', line)
                    if match:
                        academic_year = match.group(0)
                        break

        # Load the actual data table
        # Skip the first 17 rows of metadata to reach the header (UKPRN, HE Provider, etc.)
        df = pd.read_csv(file_path, skiprows=17, encoding='utf-8-sig')
```

```
        # Drop empty rows or footer notes
        df = df.dropna(subset=['HE provider'])

        # Clean numeric columns (HESA often includes commas like "1,200")
        cols_to_fix = ['Female', 'Male', 'Unknown', 'Total']
        for col in cols_to_fix:
            # Convert to string, remove commas/non-digits, and convert to
integer
            df[col]   =   df[col].astype(str).str.replace(r'[^\d]',   '',
regex=True)
            df[col]                =                pd.to_numeric(df[col],
errors='coerce').fillna(0).astype(int)

        # Add the Year column extracted earlier
        df['Academic_Year'] = academic_year

        all_data.append(df)

    # Combine all DataFrames into one
    master_df = pd.concat(all_data, ignore_index=True)

    # Final Clean: Standardize University Names to uppercase for easier
matching later
    master_df['HE           provider']           =           master_df['HE
provider'].str.strip().str.upper()

    return master_df

# Run the function
merged_data = clean_hesa_enrollments("Total Undergrad Enrollments *.csv")

# Save the final cleaned version
merged_data.to_csv("cleaned_london_enrollments_2020_2024.csv",
index=False)
print("Success! File saved as 'cleaned_london_enrollments_2020_2024.csv'")


# In[3]:


import pandas as pd

df = pd.read_csv('cleaned_london_enrollments_by_sex_2020_2024.csv')

df.head()


# In[4]:


import pandas as pd
import glob
import re
```

```
import os

def clean_hesa_enrollments_fixed(file_pattern):
    all_data = []
    files = glob.glob(file_pattern)

    for file_path in files:
        # Targeted Extraction of Year from Line 14
        academic_year = "Unknown"
        with open(file_path, 'r', encoding='utf-8-sig') as f:
            lines = f.readlines()
            # Look specifically at metadata line 14
            if len(lines) >= 14:
                year_line = lines[13]
                match = re.search(r'\d{4}/\d{2}', year_line)
                if match:
                    academic_year = match.group(0)

        # Load the data table (skipping the 17 metadata rows)
        df = pd.read_csv(file_path, skiprows=17, encoding='utf-8-sig')

        # Basic Cleaning
        df = df.dropna(subset=['HE provider'])
        for col in ['Female', 'Male', 'Unknown', 'Total']:
            df[col]   =   df[col].astype(str).str.replace(r'[^\d]',   '',
regex=True)
            df[col]                =                pd.to_numeric(df[col],
errors='coerce').fillna(0).astype(int)

        # Assign the correct year
        df['Academic_Year'] = academic_year
        all_data.append(df)
        print(f"Processed {file_path}: Found Year {academic_year}")

    master_df = pd.concat(all_data, ignore_index=True)
    master_df['HE           provider']           =           master_df['HE
provider'].str.strip().str.upper()
    return master_df

# Execute and Save
merged_data  =  clean_hesa_enrollments_fixed("Total  Undergrad  Enrollments
*.csv")
merged_data.to_csv("cleaned_london_enrollments_2020_2024.csv",
index=False)


# In[6]:


df = pd.read_csv('cleaned_london_enrollments_by_sex_2020_2024.csv')
df.head()


# In[7]:
```

```
import pandas as pd
import glob
import re
import os

def clean_hesa_residence(file_pattern):
    """
    Cleans and merges multiple HESA residence CSV files.
    """
    all_data = []

    # Find all files matching the pattern
    files = glob.glob(file_pattern)

    for file_path in files:
        # Targeted Extraction of Year from Line 14
        academic_year = "Unknown"
        with open(file_path, 'r', encoding='utf-8-sig') as f:
            lines = f.readlines()
            if len(lines) >= 14:
                year_line = lines[13]
                match = re.search(r'\d{4}/\d{2}', year_line)
                if match:
                    academic_year = match.group(0)

        # Load the data table (skipping the 17 metadata rows)
        df = pd.read_csv(file_path, skiprows=17, encoding='utf-8-sig')

        # Clean column names (remove leading/trailing spaces like "Total UK
")
        df.columns = df.columns.str.strip()

        # Drop empty rows
        df = df.dropna(subset=['HE provider'])

        # Clean all numeric columns (any column that's not UKPRN or HE
provider)
        cols_to_clean = [col for col in df.columns if col not in ['UKPRN',
'HE provider']]
        for col in cols_to_clean:
            # Convert to string, remove commas and symbols, and convert back
to integer
            df[col]    =    df[col].astype(str).str.replace(r'[^\d]',    '',
regex=True)
            df[col]                 =                pd.to_numeric(df[col],
errors='coerce').fillna(0).astype(int)

        # Assign the correct academic year and add to list
        df['Academic_Year'] = academic_year
        all_data.append(df)
        print(f"Processed: {file_path} -> Found Year: {academic_year}")
```

```
    # Combine everything into one Master DataFrame
    master_df = pd.concat(all_data, ignore_index=True)

    # Standardize University names for merging later
    master_df['HE           provider']           =           master_df['HE
provider'].str.strip().str.upper()

    return master_df

# Run the cleaning process
residence_merged = clean_hesa_residence("Enrollment by Permanent Residence
*.csv")

# Save the final cleaned file
residence_merged.to_csv("cleaned_london_enrollments_by_residence_2020_2024
.csv", index=False)


# In[8]:


df = pd.read_csv('cleaned_london_enrollments_by_residence_2020_2024.csv')
df.head()


# In[9]:


import pandas as pd
import glob
import re
import os

def clean_hesa_qualifications(file_pattern):
    """
    Cleans and merges HESA Qualifications by Classification CSV files.
    """
    all_data = []

    # Find all matching files
    files = glob.glob(file_pattern)

    for file_path in files:
        # Targeted Extraction of Year from Line 11
        academic_year = "Unknown"
        with open(file_path, 'r', encoding='utf-8-sig') as f:
            lines = f.readlines()
            if len(lines) >= 11:
                year_line = lines[10] # Line 11 is index 10
                match = re.search(r'\d{4}/\d{2}', year_line)
                if match:
                    academic_year = match.group(0)

        # Load the data table (skipping the 14 metadata rows)
```

```
        df = pd.read_csv(file_path, skiprows=14, encoding='utf-8-sig')

        # Strip whitespace from column names
        df.columns = df.columns.str.strip()

        # Drop empty rows or rows missing a provider name
        df = df.dropna(subset=['HE provider'])

        # Clean numeric columns (First class, Second class, etc.)
        # Targets every column that isn't 'UKPRN' or 'HE provider'
        cols_to_clean = [col for col in df.columns if col not in ['UKPRN',
'HE provider']]
        for col in cols_to_clean:
            # Handle commas, symbols and convert to integer
            df[col]    =    df[col].astype(str).str.replace(r'[^\d]',    '',
regex=True)
            df[col]                =                pd.to_numeric(df[col],
errors='coerce').fillna(0).astype(int)

        # Add the Year column
        df['Academic_Year'] = academic_year
        all_data.append(df)
        print(f"Processed: {file_path} -> Year {academic_year}")

    # Combine all years into one Master DataFrame
    master_df = pd.concat(all_data, ignore_index=True)

    # Standardize University Names (uppercase and trimmed)
    master_df['HE           provider']           =           master_df['HE
provider'].str.strip().str.upper()

    return master_df

qualifications_merged   =   clean_hesa_qualifications("Qualifications   by
Classification *.csv")

qualifications_merged.to_csv("cleaned_london_qualifications_2020_2024.csv"
, index=False)


# In[11]:


df                                                                      =
pd.read_csv('cleaned_london_qualifications_by_classification_2020_2024.csv
')
df.head()


# In[12]:


import pandas as pd
import glob
```

```
import re

def clean_hesa_agriculture(file_pattern):
    """
    Cleans and merges HESA Agriculture and Food studies enrollment CSVs.
    """
    all_data = []
    files = sorted(glob.glob(file_pattern))

    for file_path in files:
        # Identify Academic Year from filters (searching metadata block)
        academic_year = "Unknown"
        with open(file_path, 'r', encoding='utf-8-sig') as f:
            lines = f.readlines()
            # Skip the first few lines to avoid the date range in the
Subtitle
            for line in lines[8:20]:
                if "Academic year" in line:
                    match = re.search(r'\d{4}/\d{2}', line)
                    if match:
                        academic_year = match.group(0)
                        break

        # Load the data table (skipping the 16 metadata rows)
        df = pd.read_csv(file_path, skiprows=16, encoding='utf-8-sig')

        # Clean column names (strip whitespace)
        df.columns = df.columns.str.strip()

        # Drop empty rows
        df = df.dropna(subset=['HE provider'])

        # Clean numeric columns (Animal science, Food sciences, etc.)
        cols_to_clean = [col for col in df.columns if col not in ['UKPRN',
'HE provider']]
        for col in cols_to_clean:
            df[col]   =   df[col].astype(str).str.replace(r'[^\d]',   '',
regex=True)
            df[col]                =                pd.to_numeric(df[col],
errors='coerce').fillna(0).astype(int)

        # Add the Academic Year column
        df['Academic_Year'] = academic_year
        all_data.append(df)
        print(f"Processed: {file_path} -> Year {academic_year}")

    # Combine into Master DataFrame
    master_df = pd.concat(all_data, ignore_index=True)

    # Standardize University Names
    master_df['HE          provider']          =          master_df['HE
provider'].str.strip().str.upper()

    return master_df
```

```
# Run the cleaning function
agri_cleaned = clean_hesa_agriculture("Enrollment Agriculture, Food and
Related *.csv")

agri_cleaned.to_csv("cleaned_london_enrollments_for_agriculture_2020_2024.
csv", index=False)


# In[14]:


df                                                                    =
pd.read_csv('cleaned_london_enrollments_for_agriculture_2020_2024.csv')
df.head()


# In[15]:


import pandas as pd
import glob
import re

def clean_hesa_architecture(file_pattern, output_name):
    """
    Cleans and merges HESA Architecture, Building and Planning enrollment
CSVs.
    """
    all_data = []
    # Identify all files matching the pattern and sort them
    files = sorted(glob.glob(file_pattern))

    for file_path in files:
        # Extract Academic Year from the metadata section
        academic_year = "Unknown"
        with open(file_path, 'r', encoding='utf-8-sig') as f:
            lines = f.readlines()
            for line in lines[8:25]:  # Look specifically within the filter
block
                if "Academic year" in line:
                    match = re.search(r'\d{4}/\d{2}', line)
                    if match:
                        academic_year = match.group(0)
                        break

        # Load the data table (skip the first 16 metadata rows)
        df = pd.read_csv(file_path, skiprows=16, encoding='utf-8-sig')

        # Clean column names
        df.columns = df.columns.str.strip()

        # Drop empty rows
        df = df.dropna(subset=['HE provider'])
```

```
        # Clean numeric columns (e.g., "13-01-01 Architecture", "13-01-02
Building")
        cols_to_clean = [col for col in df.columns if col not in ['UKPRN',
'HE provider']]
        for col in cols_to_clean:
            # Remove commas and non-numeric characters (like privacy
symbols)
            df[col]   =   df[col].astype(str).str.replace(r'[^\d]',   '',
regex=True)
            df[col]                =                pd.to_numeric(df[col],
errors='coerce').fillna(0).astype(int)

        # Add the Academic Year as a column
        df['Academic_Year'] = academic_year
        all_data.append(df)
        print(f"Successfully    processed:    {file_path}    ->    Year
{academic_year}")

    # Combine all yearly dataframes into one
    master_df = pd.concat(all_data, ignore_index=True)

    # Standardize HE provider names to UPPERCASE for easier merging later
    master_df['HE           provider']           =           master_df['HE
provider'].str.strip().str.upper()

    # Save the final cleaned dataset
    master_df.to_csv(output_name, index=False)
    return master_df

# Execute and create the master file
arch_cleaned  =  clean_hesa_architecture("Enrollment  for  Architecture,
Building and Planning *.csv",

"cleaned_london_enrollments_for_architecture_2020_2025.csv")


# In[16]:


import pandas as pd
import re

# List of files to be batch cleaned
files = [
    'Enrollment for Biological and Sport Sciences 2020-2021.csv',
    'Enrollment for Biological and Sport Sciences 2021-2022.csv',
    'Enrollment for Biological and Sport Sciences 2022-2023.csv',
    'Enrollment for Biological and Sport Sciences 2023-2024.csv',
    'Enrollment for Biological and Sport Sciences 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
```

```
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        # Taking last two digits of second year for standard HESA format
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    # Skip first 16 metadata rows
    # Used 'utf-8-sig' to handle potential byte-order marks in CSVs
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    # Append the Academic Year column
    df['Academic Year'] = extract_academic_year(file)

    # Identify numeric columns for cleaning (columns except UKPRN, HE
provider, and Academic Year)
    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        # 1. Convert everything to string
        # 2. Remove commas
        # 3. Convert to numeric (replace non-numeric entries with 0)
        df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

# Concatenate all years into one master DataFrame
master_df = pd.concat(dfs, ignore_index=True)

# Save the final cleaned dataset
master_df.to_csv('cleaned_london_enrollments_for_biological_and_sport_scie
nces_2020_2024.csv', index=False)

print("Process Complete. File saved as:
cleaned_london_enrollments_for_biological_and_sport_sciences_2020_2024.csv
")
print(f"Total rows processed: {len(master_df)}")


# In[17]:


df =
pd.read_csv('cleaned_london_enrollments_for_biological_and_sport_sciences_
2020_2024.csv')
df.head()


# In[19]:
```

```
import pandas as pd
import re

# List of files for batch cleaning
files = [
    'Enrollments for Business and Management 2020-2021.csv',
    'Enrollments for Business and Management 2021-2022.csv',
    'Enrollments for Business and Management 2022-2023.csv',
    'Enrollments for Business and Management 2023-2024.csv',
    'Enrollments for Business and Management 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        # Taking last two digits of second year for standard HESA format
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    # Skip first 16 metadata rows
    # Used 'utf-8-sig' to handle potential byte-order marks in CSVs
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    # Append the Academic Year column
    df['Academic Year'] = extract_academic_year(file)

    # Identify numeric columns for cleaning (columns except UKPRN, HE
provider, and Academic Year)
    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        # 1. Convert everything to string
        # 2. Remove commas
        # 3. Convert to numeric (replace non-numeric entries with 0)
        df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

# Concatenate all years into one master DataFrame
master_df = pd.concat(dfs, ignore_index=True)

# Save the final cleaned dataset
master_df.to_csv('cleaned_london_enrollments_for_business_and_management_2
020_2025.csv', index=False)
```

```
print("Process Complete. File saved as: cleaned_london_enrollments_for_business_and_management_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[20]:


import pandas as pd
import re

files = [
    'Enrollments for Combined and General Studies 2020-2021.csv',
    'Enrollments for Combined and General Studies 2021-2022.csv',
    'Enrollments for Combined and General Studies 2022-2023.csv',
    'Enrollments for Combined and General Studies 2023-2024.csv',
    'Enrollments for Combined and General Studies 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', '', regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_combined_studies_2020_2025.csv', index=False)

print("Process Complete. File saved as: cleaned_london_enrollments_for_combined_studies_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[21]:
```

```
import pandas as pd
import re

files = [
    'Enrollment for Computing 2020-2021.csv',
    'Enrollment for Computing 2021-2022.csv',
    'Enrollment for Computing 2022-2023.csv',
    'Enrollment for Computing 2023-2024.csv',
    'Enrollment for Computing 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col]  =  pd.to_numeric(df[col].astype(str).str.replace(',',  '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_computing_2020_2025.csv',
index=False)

print("Process          Complete.          File          saved          as:
cleaned_london_enrollments_for_computing_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[22]:


import pandas as pd
import re

files = [
```

```
    'Enrollments for Design, Creative and Performing Arts 2020-2021.csv',
    'Enrollments for Design, Creative and Performing Arts 2021-2022.csv',
    'Enrollments for Design, Creative and Performing Arts 2022-2023.csv',
    'Enrollments for Design, Creative and Performing Arts 2023-2024.csv',
    'Enrollments for Design, Creative and Performing Arts 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col]  =  pd.to_numeric(df[col].astype(str).str.replace(',',  '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_design_and_performing_art
s_2020_2025.csv', index=False)

print("Process          Complete.          File          saved          as:
cleaned_london_enrollments_for_design_and_performing_arts_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[23]:


import pandas as pd
import re

# List of your uploaded files
files = [
    'Enrollments for Education and Teaching 2020-2021.csv',
    'Enrollments for Education and Teaching 2021-2022.csv',
    'Enrollments for Education and Teaching 2022-2023.csv',
    'Enrollments for Education and Teaching 2023-2024.csv',
    'Enrollments for Education and Teaching 2024-2025.csv'
```

```
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        # Taking last two digits of second year for standard HESA format
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    # Read data: skip first 16 metadata rows
    # Use 'utf-8-sig' to handle potential byte-order marks in CSVs
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    # Append the Academic Year column
    df['Academic Year'] = extract_academic_year(file)

    # Identify numeric columns for cleaning (columns except UKPRN, HE
provider, and Academic Year)
    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        # 1. Convert everything to string
        # 2. Remove commas (e.g., "1,250" -> "1250")
        # 3. Convert to numeric (replace non-numeric entries with 0)
        df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

# Concatenate all years into one master DataFrame
master_df = pd.concat(dfs, ignore_index=True)

# Save the final cleaned dataset
master_df.to_csv('cleaned_london_enrollments_for_education_and_teaching_20
20_2025.csv', index=False)

print("Process Complete. File saved as:
cleaned_london_enrollments_for_education_and_teaching_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[24]:


import pandas as pd
import re

files = [
```

```
    'Enrollments for Engineering and Technology 2020-2021.csv',
    'Enrollments for Engineering and Technology 2021-2022.csv',
    'Enrollments for Engineering and Technology 2022-2023.csv',
    'Enrollments for Engineering and Technology 2023-2024.csv',
    'Enrollments for Engineering and Technology 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col]  =  pd.to_numeric(df[col].astype(str).str.replace(',',  '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_engineering_and_technolog
y_2020_2025.csv', index=False)

print("Process          Complete.          File          saved          as:
cleaned_london_enrollments_for_engineering_and_technology_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[25]:


import pandas as pd
import re

files = [
    'EN93CD~1.CSV',
    'ENROLL~1.CSV',
    'ENROLL~2.CSV',
    'ENROLL~3.CSV',
    'ENROLL~4.CSV'
]
```

```
def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col]  =  pd.to_numeric(df[col].astype(str).str.replace(',',  '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_geography_and_environment
_social_science_2020_2025.csv', index=False)

print("Process          Complete.          File          saved          as:
cleaned_london_enrollments_for_geography_and_environment_social_science_20
20_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[26]:


import pandas as pd
import re

files = [
    'Enrollments for Geography, Earth and Environment 2020-2021.csv',
    'Enrollments for Geography, Earth and Environment 2021-2022.csv',
    'Enrollments for Geography, Earth and Environment 2022-2023.csv',
    'Enrollments for Geography, Earth and Environment 2023-2024.csv',
    'Enrollments for Geography, Earth and Environment 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
```

```
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col]  =  pd.to_numeric(df[col].astype(str).str.replace(',',  '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_geography_and_environment
_natural_science_2020_2025.csv', index=False)

print("Process          Complete.          File          saved          as:
cleaned_london_enrollments_for_geography_and_environment_natural_science_2
020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[1]:


import pandas as pd
import re

files = [
    'Enrollments for Historical and Religious Studies 2020-2021.csv',
    'Enrollments for Historical and Religious Studies 2021-2022.csv',
    'Enrollments for Historical and Religious Studies 2022-2023.csv',
    'Enrollments for Historical and Religious Studies 2023-2024.csv',
    'Enrollments for Historical and Religious Studies 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"
```

```
dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col]  =  pd.to_numeric(df[col].astype(str).str.replace(',',  '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_historical_and_religious_
studies_2020_2025.csv', index=False)

print("Process          Complete.          File          saved          as:
cleaned_london_enrollments_for_historical_and_religious_studies_2020_2025.
csv")
print(f"Total rows processed: {len(master_df)}")


# In[2]:


import pandas as pd
import re

files = [
    'Enrollments for Language and Area Studies 2020-2021.csv',
    'Enrollments for Language and Area Studies 2021-2022.csv',
    'Enrollments for Language and Area Studies 2022-2023.csv',
    'Enrollments for Language and Area Studies 2023-2024.csv',
    'Enrollments for Language and Area Studies 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)
```

```
    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col]  =  pd.to_numeric(df[col].astype(str).str.replace(',',  '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_language_and_area_studies
_2020_2025.csv', index=False)

print("Process          Complete.          File          saved          as:
cleaned_london_enrollments_for_language_and_area_studies_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[3]:


import pandas as pd
import re

files = [
    'Enrollments for Law 2020-2021.csv',
    'Enrollments for Law 2021-2022.csv',
    'Enrollments for Law 2022-2023.csv',
    'Enrollments for Law 2023-2024.csv',
    'Enrollments for Law 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
```

```
        df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_law_2020_2025.csv',
index=False)

print("Process Complete. File saved as:
cleaned_london_enrollments_for_law_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[4]:


import pandas as pd
import re

files = [
    'Enrollments for Mathematical Sciences 2020-2021.csv',
    'Enrollments for Mathematical Sciences 2021-2022.csv',
    'Enrollments for Mathematical Sciences 2022-2023.csv',
    'Enrollments for Mathematical Sciences 2023-2024.csv',
    'Enrollments for Mathematical Sciences 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)
```

```
master_df.to_csv('cleaned_london_enrollments_for_mathematical_sciences_2020_2025.csv', index=False)

print("Process Complete. File saved as: cleaned_london_enrollments_for_mathematical_sciences_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[5]:


import pandas as pd
import re

files = [
    'Enrollments for Media, Journalism and Communications 2020-2021.csv',
    'Enrollments for Media, Journalism and Communications 2021-2022.csv',
    'Enrollments for Media, Journalism and Communications 2022-2023.csv',
    'Enrollments for Media, Journalism and Communications 2023-2024.csv',
    'Enrollments for Media, Journalism and Communications 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', '', regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_media_journalism_and_communications_2020_2025.csv', index=False)
```

```
print("Process Complete. File saved as: cleaned_london_enrollments_for_media_journalism_and_communications_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[6]:


import pandas as pd
import re

files = [
    'Enrollments for Medicine and Dentistry 2020-2021.csv',
    'Enrollments for Medicine and Dentistry 2021-2022.csv',
    'Enrollments for Medicine and Dentistry 2022-2023.csv',
    'Enrollments for Medicine and Dentistry 2023-2024.csv',
    'Enrollments for Medicine and Dentistry 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    df['Academic Year'] = extract_academic_year(file)

    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE provider', 'Academic Year']]

    for col in cols_to_clean:
        df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', '', regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

master_df = pd.concat(dfs, ignore_index=True)

master_df.to_csv('cleaned_london_enrollments_for_medicine_and_dentistry_2020_2025.csv', index=False)

print("Process Complete. File saved as: cleaned_london_enrollments_for_medicine_and_dentistry_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")
```

```
# In[11]:


import pandas as pd
import re
import glob

# List of subjects to process
subject_areas = [
    "Physical Sciences",
    "Psychology",
    "Social Sciences",
    "Subjects Allied to Medicine",
    "Veterinary Sciences"
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

# Loop through each subject area
for subject in subject_areas:
    print(f"Processing: {subject}...")

    # Automatically finds all CSV files containing the subject name
    file_list = glob.glob(f"*{subject}*.csv")

    if not file_list:
        print(f"  - No files found for {subject}. Skipping.")
        continue

    dfs = []
    for file in file_list:
        try:
            # Read the data and skip the first 16 metadata rows
            df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

            # Add Academic Year and Subject Area columns
            df['Academic Year'] = extract_academic_year(file)
            df['Subject Area'] = subject

            # Identify numeric columns for cleaning
            cols_to_clean = [col for col in df.columns if col not in
['UKPRN', 'HE provider', 'Academic Year', 'Subject Area']]

            for col in cols_to_clean:
                df[col]                                                 =
pd.to_numeric(df[col].astype(str).str.replace(',',     '',     regex=True),
errors='coerce').fillna(0).astype(int)
```

```
            dfs.append(df)
            print(f"  - Successfully loaded: {file}")
        except Exception as e:
            print(f"  - Error processing {file}: {e}")

    # Concatenate and save if files were found
    if dfs:
        master_df = pd.concat(dfs, ignore_index=True)

        #        Create        a        clean        filename        (e.g.,
"cleaned_biological_and_sport_sciences.csv")
        clean_name = subject.lower().replace(' ', '_').replace(',', '')
        output_filename = f"cleaned_london_{clean_name}_2020_2025.csv"

        master_df.to_csv(output_filename, index=False)
        print(f"Done! Saved as: {output_filename}\n")

print("All subjects in the list have been processed.")


# In[12]:


import pandas as pd
import re

# List of files to clean. They had a slightly different format than previous
subject files.
files = [
    'Enrollments by Study Area 2020-2021.csv',
    'Enrollments by Study Area 2021-2022.csv',
    'Enrollments by Study Area 2022-2023.csv',
    'Enrollments by Study Area 2023-2024.csv',
    'Enrollments by Study Area 2024-2025.csv'
]

def extract_academic_year(filename):
    """Helper to convert '2020-2021' to '2020/21' format"""
    match = re.search(r'(\d{4})-(\d{4})', filename)
    if match:
        start_year = match.group(1)
        # Taking last two digits of second year for standard HESA format
        end_year = match.group(2)[-2:]
        return f"{start_year}/{end_year}"
    return "Unknown"

dfs = []
for file in files:
    # Read data and skip first 16 metadata rows
    df = pd.read_csv(file, skiprows=16, encoding='utf-8-sig')

    # Append the Academic Year column
    df['Academic Year'] = extract_academic_year(file)
```

```
    # Identify numeric columns for cleaning (columns except UKPRN, HE
provider, and Academic Year)
    cols_to_clean = [col for col in df.columns if col not in ['UKPRN', 'HE
provider', 'Academic Year']]

    for col in cols_to_clean:
        # 1. Convert everything to string
        # 2. Remove commas
        # 3. Convert to numeric (replace non-numeric entries with 0)
        df[col]  =  pd.to_numeric(df[col].astype(str).str.replace(',',  '',
regex=True), errors='coerce').fillna(0).astype(int)

    dfs.append(df)

# Concatenate all years into one master DataFrame
master_df = pd.concat(dfs, ignore_index=True)

# Save the final cleaned dataset
master_df.to_csv('cleaned_london_enrollments_by_study_area_2020_2025.csv',
index=False)

print("Process          Complete.          File          saved          as:
cleaned_london_enrollments_by_study_area_2020_2025.csv")
print(f"Total rows processed: {len(master_df)}")


# In[4]:


import pandas as pd
import re
import glob

# files for processing. UKPRN is going to be extracted from reference_file
reference_file = 'cleaned_london_enrollments_by_study_area_2020_2025.csv'
employment_files = [
    'EMPLOYMENT2020.csv', 'EMPLOYMENT2021.csv', 'EMPLOYMENT2022.csv',
    'EMPLOYMENT2023.csv', 'EMPLOYMENT2024.csv'
]

# Load the specific reference file to extract London UKPRNs
ref_df = pd.read_csv(reference_file)
london_ukprns = ref_df['UKPRN'].dropna().unique().astype(int)
print(f"Identified {len(london_ukprns)} London UKPRNs for filtering.")

all_employment_dfs = []

# Loop through Employment files
for file in employment_files:
    # Extract the year from the filename
    year_match = re.search(r'(\d{4})', file)
    year = year_match.group(1) if year_match else "Unknown"
```

```
    # Load and filter by UKPRN
    df = pd.read_csv(file, low_memory=False)
    df['UKPRN'] = pd.to_numeric(df['UKPRN'], errors='coerce')
    df_london = df[df['UKPRN'].isin(london_ukprns)].copy()

    # Add metadata
    df_london['Academic_Year'] = year

    # Clean numeric metrics
    # This handles commas, percentages, and suppressed values ('.', '*')
    numeric_cols = ['WORK', 'UNEMP', 'WORKSTUDY', 'STUDY', 'BOTH',
'NOAVAIL', 'EMPPOP']
    for col in numeric_cols:
        if col in df_london.columns:
            df_london[col]                                                =
pd.to_numeric(df_london[col].astype(str).str.replace(r'[^\d.]',        '',
regex=True), errors='coerce').fillna(0)

    all_employment_dfs.append(df_london)
    print(f"  - {file}: Found {len(df_london)} London course entries.")

# Merge all years into one Master File
merged_employment = pd.concat(all_employment_dfs, ignore_index=True)
merged_employment.to_csv('cleaned_london_employment_2020_2024.csv',
index=False)


# In[5]:


import pandas as pd
import re

reference_file = 'cleaned_london_enrollments_by_study_area_2020_2025.csv'
entry_files = [
    'ENTRY2020.csv', 'ENTRY2021.csv', 'ENTRY2022.csv',
    'ENTRY2023.csv', 'ENTRY2024.csv'
]

# Load HESA Master file to extract the 123 London UKPRNs
ref_df = pd.read_csv(reference_file)
london_ukprns = ref_df['UKPRN'].dropna().unique().astype(int)

all_entry_dfs = []

# Loop through each Entry file
for file in entry_files:
    # Extract the year from the filename
    year_match = re.search(r'(\d{4})', file)
    year = year_match.group(1) if year_match else "Unknown"

    # Load and filter by UKPRN
    df = pd.read_csv(file, low_memory=False)
    df['UKPRN'] = pd.to_numeric(df['UKPRN'], errors='coerce')
```

```
    df_london = df[df['UKPRN'].isin(london_ukprns)].copy()

    # Add metadata
    df_london['Academic_Year'] = year

    # Clean numeric qualification columns (percentages)
    qual_cols = ['ACCESS', 'ALEVEL', 'BACC', 'DEGREE', 'FOUNDTN', 'NOQUALS',
'OTHER', 'OTHERHE']
    for col in qual_cols:
        if col in df_london.columns:
            df_london[col]                                                =
pd.to_numeric(df_london[col].astype(str).str.replace(r'[^\d.]',        '',
regex=True), errors='coerce').fillna(0)

    all_entry_dfs.append(df_london)
    print(f"Processed {file}: Found {len(df_london)} records.")

# Merge all years into one Master Entry file
merged_entry = pd.concat(all_entry_dfs, ignore_index=True)
merged_entry.to_csv('cleaned_london_entry_qualifications_2020_2024.csv',
index=False)


# In[7]:


import pandas as pd
import re

reference_file = 'cleaned_london_enrollments_by_study_area_2020_2025.csv'
nss_files = ['NSS2020.csv', 'NSS2021.csv', 'NSS2022.csv', 'NSS2023.csv',
'NSS2024.csv']

# HESA Master file to extract London UKPRNs
ref_df = pd.read_csv(reference_file)
london_ukprns = ref_df['UKPRN'].dropna().unique().astype(int)

all_nss_dfs = []

# Process each NSS CSV
for file in nss_files:
    # Extract the year from the filename
    year_match = re.search(r'(\d{4})', file)
    year = year_match.group(1) if year_match else "Unknown"

    try:
        # Load national data
        df = pd.read_csv(file, low_memory=False)
        df['UKPRN'] = pd.to_numeric(df['UKPRN'], errors='coerce')

        # Filter for London Universities only
        df_london = df[df['UKPRN'].isin(london_ukprns)].copy()
        df_london['Academic_Year'] = year
```

```
        # Clean numeric survey columns (Q1-Q27, T1-T7, and Population)
        score_cols = [f'Q{i}' for i in range(1, 28)] + [f'T{i}' for i in
range(1, 8)]
        other_metrics = ['NSSPOP', 'NSSRESP_RATE', 'NSSAGG']
        target_cols = score_cols + other_metrics

        for col in target_cols:
            if col in df_london.columns:
                df_london[col]                                                =
pd.to_numeric(df_london[col].astype(str).str.replace(r'[^\d.]',        '',
regex=True), errors='coerce').fillna(0)

        all_nss_dfs.append(df_london)
        print(f"Processed  {file}:  Found  {len(df_london)}  London  course
records.")

    except Exception as e:
        print(f"Error processing {file}: {e}")

# Merge all years into one Master NSS file
merged_nss = pd.concat(all_nss_dfs, ignore_index=True)
merged_nss.to_csv('cleaned_london_nss_2020_2024.csv', index=False)


# In[8]:


import pandas as pd
import re

reference_file = 'cleaned_london_enrollments_by_study_area_2020_2025.csv'
tariff_files  =  ['TARIFF2020.csv',  'TARIFF2021.csv',  'TARIFF2022.csv',
'TARIFF2023.csv', 'TARIFF2024.csv']

# Load HESA file to extract the 123 London UKPRNs
ref_df = pd.read_csv(reference_file)
london_ukprns = ref_df['UKPRN'].dropna().unique().astype(int)

all_tariff_dfs = []

# Process each Tariff CSV
for file in tariff_files:
    # Extract the year from the filename
    year_match = re.search(r'(\d{4})', file)
    year = year_match.group(1) if year_match else "Unknown"

    try:
        # Load national data
        df = pd.read_csv(file, low_memory=False)
        df['UKPRN'] = pd.to_numeric(df['UKPRN'], errors='coerce')

        # Filter for London Universities only
        df_london = df[df['UKPRN'].isin(london_ukprns)].copy()
        df_london['Academic_Year'] = year
```

```
        # Clean numeric tariff columns (T001, T048... up to T240)
        tariff_cols = [f'T{str(i).zfill(3)}' for i in [1, 48, 64, 80, 96,
112, 128, 144, 160, 176, 192, 208, 224, 240]]
        other_metrics = ['TARPOP', 'TARAGG']
        target_cols = tariff_cols + other_metrics

        for col in target_cols:
            if col in df_london.columns:
                # Remove symbols and handle suppressed values
                df_london[col]                                          =
pd.to_numeric(df_london[col].astype(str).str.replace(r'[^\d.]',        '',
regex=True), errors='coerce').fillna(0)

        all_tariff_dfs.append(df_london)
        print(f"Processed {file}: Found {len(df_london)} records.")

    except Exception as e:
        print(f"Error processing {file}: {e}")

# Merge all years into one Master Tariff file
merged_tariff = pd.concat(all_tariff_dfs, ignore_index=True)
merged_tariff.to_csv('cleaned_london_tariff_2020_2024.csv', index=False)


# In[14]:


import pandas as pd
import re
import os

# Set the filename
file_name = "Rankings 2020-2024.xlsx"

def load_ranking_data(file):
    """Detects file type and loads the ranking table."""
    ext = os.path.splitext(file)[1].lower()

    if ext == '.xlsx':
        # Reads Excel
        return pd.read_excel(file, skiprows=2, header=None)
    elif ext == '.csv':
        # Reads CSV
        return pd.read_csv(file, skiprows=2, header=None)
    else:
        raise ValueError("Unsupported file format. Please use .xlsx or
.csv")

try:
    # Load the data
    df = load_ranking_data(file_name)

    # Define clean column names manually
```

```
    cols = [
        'UNIVERSITY', 'UKPRN',
        '2020_QS', '2020_THE',
        '2021_QS', '2021_THE',
        '2022_QS', '2022_THE',
        '2023_QS', '2023_THE',
        '2024_QS', '2024_THE'
    ]
    df.columns = cols

    # Reshape from Wide to Long format
    df_melted       =       df.melt(id_vars=['UNIVERSITY',       'UKPRN'],
var_name='Year_Source', value_name='Rank')

    # Extract Year and Source (e.g., '2020' and 'QS')
    df_melted[['Year', 'Source']] = df_melted['Year_Source'].str.split('_',
expand=True)
    df_melted.drop(columns=['Year_Source'], inplace=True)

    # Pivot so QS and THE are separate columns
    df_tidy = df_melted.pivot_table(index=['UNIVERSITY', 'UKPRN', 'Year'],
                                    columns='Source',
                                    values='Rank',
                                    aggfunc='first').reset_index()

    # Clean numeric ranks (taking the lower bound for ranges like "351-
400")
    def to_numeric_rank(val):
        if pd.isna(val) or str(val).lower() in ['null', 'none', '']:
            return None
        match = re.search(r'(\d+)', str(val))
        return int(match.group(1)) if match else None

    df_tidy['QS_Rank'] = df_tidy['QS'].apply(to_numeric_rank)
    df_tidy['THE_Rank'] = df_tidy['THE'].apply(to_numeric_rank)

    # Format Academic Year to match HESA format (e.g. "2020/21")
    df_tidy['Academic_Year']   =   df_tidy['Year'].astype(str)   +   "/"   +
(df_tidy['Year'].astype(int) + 1).astype(str).str[-2:]

    # Save the final cleaned data
    df_tidy.to_csv('cleaned_london_university_rankings_2020_2024.csv',
index=False)
    print("Success!        Cleaned        rankings        saved        to
'cleaned_london_university_rankings_2020_2024.csv'")

except Exception as e:
    print(f"Error: {e}")
    print("Check if the 'file_name' variable matches your file name exactly
(including .xlsx or .csv)")


# In[18]:
```

```
import pandas as pd
import numpy as np

# Define files to be merged
file_map = {
    'nss': "cleaned_london_nss_2020_2024.csv",
    'employment': "cleaned_london_employment_2020_2024.csv",
    'entry': "cleaned_london_entry_qualifications_2020_2024.csv",
    'tariff': "cleaned_london_tariff_2020_2024.csv",
    'rankings': "cleaned_london_university_rankings_2020_2024.csv",
    'residence': "cleaned_london_enrollments_by_residence_2020_2024.csv",
    'sex': "cleaned_london_enrollments_by_sex_2020_2024.csv",
    'classification':
"cleaned_london_qualifications_by_classification_2020_2024.csv",
    'study_area':
"cleaned_london_enrollments_by_study_area_2020_2025.csv",
    'agriculture':
"cleaned_london_enrollments_for_agriculture_2020_2024.csv"
}

def standardize_year(y):
    y = str(y)
    if '/' in y: return y
    if '-' in y: return y.replace('-', '/')
    try:
        y_int = int(y)
        return f"{y_int}/{str(y_int + 1)[-2:]}"
    except:
        return y

# Load all dataframes with low_memory=False
dfs = {}
for key, filename in file_map.items():
    df = pd.read_csv(filename, low_memory=False)
    if 'Academic Year' in df.columns: df.rename(columns={'Academic Year':
'Academic_Year'}, inplace=True)
    if    'Academic_Year'    in    df.columns:    df['Academic_Year']    =
df['Academic_Year'].apply(standardize_year)
    dfs[key] = df

# Merge Course-Level Data (NSS, Employment, Entry, Tariff)
course_keys = ['UKPRN', 'KISCOURSEID', 'KISMODE', 'Academic_Year']
master = dfs['nss']
for key in ['employment', 'entry', 'tariff']:
    master  =  pd.merge(master,  dfs[key],  on=course_keys,  how='outer',
suffixes=('', f'_{key}'))

# Clean redundant columns
master = master.loc[:, ~master.columns.duplicated()]

# Aggregate University-Level Context
uni_keys = ['UKPRN', 'Academic_Year']
```

```
res_ctx = dfs['residence'][uni_keys + ['Total UK', 'Total Non-UK',
'Total']].rename(columns={'Total': 'Uni_Total_Enrollment'})
sex_ctx = dfs['sex'][uni_keys + ['Female',
'Male']].rename(columns={'Female': 'Uni_Female', 'Male': 'Uni_Male'})
rank_ctx = dfs['rankings'][uni_keys + ['QS_Rank', 'THE_Rank']]
class_ctx = dfs['classification'][uni_keys + ['First class honours', 'Upper
second class honours']]

# Combine university contexts
uni_master = pd.merge(rank_ctx, res_ctx, on=uni_keys, how='outer')
uni_master = pd.merge(uni_master, sex_ctx, on=uni_keys, how='outer')
uni_master = pd.merge(uni_master, class_ctx, on=uni_keys, how='outer')

# Final Master Join
final_master = pd.merge(master, uni_master, on=uni_keys, how='left')

# Calculate Custom Course Value Index Score
final_master['WORK'] = pd.to_numeric(final_master['WORK'],
errors='coerce').fillna(0)
final_master['Q27'] = pd.to_numeric(final_master['Q27'],
errors='coerce').fillna(0)
final_master['QS_Rank'] = pd.to_numeric(final_master['QS_Rank'],
errors='coerce')

# Prestige Score calculation (Inverting rank: 1st place = 100 points)
max_rank = final_master['QS_Rank'].max()
final_master['Prestige_Score'] = (1 -
(final_master['QS_Rank'].fillna(max_rank) / max_rank)) * 100

# Final Index: 40% Work Success, 30% NSS Satisfaction, 30% Prestige
final_master['Course_Value_Index'] = (
    (final_master['WORK'] * 0.4) +
    (final_master['Q27'] * 0.3) +
    (final_master['Prestige_Score'] * 0.3)
)

# Export Master Dataset
final_master.to_csv("COMPLETE_MASTER_COURSE_ANALYTICS.csv", index=False)


# In[21]:


import pandas as pd
import numpy as np

# Load previously created Master file
master = pd.read_csv('COMPLETE_MASTER_COURSE_ANALYTICS.csv',
low_memory=False)

# Remaining subject files
subject_files = [
    "cleaned_london_enrollments_for_architecture_2020_2025.csv",
```

```
"cleaned_london_enrollments_for_biological_and_sport_sciences_2020_2024.cs
v",

"cleaned_london_enrollments_for_business_and_management_2020_2025.csv",
    "cleaned_london_enrollments_for_combined_studies_2020_2025.csv",
    "cleaned_london_enrollments_for_computing_2020_2025.csv",

"cleaned_london_enrollments_for_design_and_performing_arts_2020_2025.csv",
    "cleaned_london_enrollments_for_education_and_teaching_2020_2025.csv",

"cleaned_london_enrollments_for_engineering_and_technology_2020_2025.csv",

"cleaned_london_enrollments_for_geography_and_environment_natural_science_
2020_2025.csv",

"cleaned_london_enrollments_for_geography_and_environment_social_science_2
020_2025.csv",

"cleaned_london_enrollments_for_historical_and_religious_studies_2020_2025
.csv",

"cleaned_london_enrollments_for_language_and_area_studies_2020_2025.csv",
    "cleaned_london_enrollments_for_law_2020_2025.csv",
    "cleaned_london_enrollments_for_mathematical_sciences_2020_2025.csv",

"cleaned_london_enrollments_for_media_journalism_and_communications_2020_2
025.csv",
    "cleaned_london_enrollments_for_medicine_and_dentistry_2020_2025.csv",
    "cleaned_london_enrollments_for_physical_sciences_2020_2025.csv",
    "cleaned_london_enrollments_for_psychology_2020_2025.csv",
    "cleaned_london_enrollments_for_social_sciences_2020_2025.csv",

"cleaned_london_enrollments_for_subjects_allied_to_medicine_2020_2025.csv"
,
    "cleaned_london_enrollments_for_veterinary_sciences_2020_2025.csv"
]

def standardize_year(y):
    y = str(y)
    if '/' in y: return y
    if '-' in y: return y.replace('-', '/')
    try:
        y_int = int(y)
        return f"{y_int}/{str(y_int + 1)[-2:]}"
    except:
        return y

# Loop and merge
for f in subject_files:
    print(f"Merging {f}...")
    sdf = pd.read_csv(f, low_memory=False)

    # Standardize Years and Columns
```

```
    if 'Academic Year' in sdf.columns:
        sdf.rename(columns={'Academic       Year':       'Academic_Year'},
inplace=True)
    sdf['Academic_Year'] = sdf['Academic_Year'].apply(standardize_year)

    # Drop columns that cause collisions
    cols_to_drop = ['Subject Area', 'HE provider']
    sdf.drop(columns=[c  for  c  in  cols_to_drop  if  c  in  sdf.columns],
inplace=True)

    # Prefix the numeric enrollment columns
    meta_keys = ['UKPRN', 'Academic_Year']
    subject_cols = [c for c in sdf.columns if c not in meta_keys]

    new_names = {}
    for c in subject_cols:
        if c[0:2].isdigit() and ' ' in c and '-' not in c.split(' ')[0]:
            new_names[c] = 'Uni_Enroll_Total_' + c
        else:
            new_names[c] = 'Uni_Enroll_Sub_' + c
    sdf.rename(columns=new_names, inplace=True)

    # Ensure keys are unique before merging
    sdf = sdf.groupby(['UKPRN', 'Academic_Year']).first().reset_index()

    # Left Merge onto master (using UKPRN and Academic_Year as the bridge)
    master   =   pd.merge(master,   sdf,   on=['UKPRN',   'Academic_Year'],
how='left')

# Final Export
master.to_csv("FINAL_COMPLETE_MASTER_DATA.csv", index=False)
print("Done! All collisions avoided.")


# In[22]:


import pandas as pd

# Target list of UKPRNs for 22 chosen universities
target_ukprns = [
    10007760, 10002718, 10003645, 10004063, 10007775, 10007780,
    10007784, 10003270, 10000961, 10003678, 10004048, 10004078, 10004351,
    10007162, 10007144, 10007146, 10007776, 10006566, 10007165, 10007843,
    10001478, 10007782
]

# Load the large master file
# Used low_memory=False to ensure course IDs and UKPRNs are read correctly
df = pd.read_csv('FINAL_COMPLETE_MASTER_DATA.csv', low_memory=False)

# Filter the rows
# Ensure UKPRN is numeric so the matching works perfectly
df['UKPRN'] = pd.to_numeric(df['UKPRN'], errors='coerce')
```

```
filtered_df = df[df['UKPRN'].isin(target_ukprns)]

# Save the smaller, filtered file
filtered_df.to_csv('FILTERED_MASTER_ANALYTICS_23_UNIS.csv', index=False)

print(f"Filtered file saved with {len(filtered_df)} rows.")


# In[29]:


import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Load the dataset
filename = 'FILTERED_MASTER_ANALYTICS_23_UNIS.csv'
df = pd.read_csv(filename, low_memory=False)

# Select the core variables used for clustering
# Prestige, Course Value, Employment (WORK), and Satisfaction (Q27)
features = ['Prestige_Score', 'Course_Value_Index', 'WORK', 'Q27']
data_subset = df[features].copy()

# Pre-processing: Fill missing values with the median
data_subset = data_subset.fillna(data_subset.median())

# Standardize the data
# This ensures that Prestige (0-100) doesn't "outweigh" other decimal
metrics
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data_subset)

# The Elbow Method Loop (k=1 to 10)
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10,
random_state=42)
    kmeans.fit(scaled_data)
    wcss.append(kmeans.inertia_)

# Plot the Elbow Curve
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS (Inertia)')
plt.show()


# In[23]:


import pandas as pd
```

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Load filtered data
df = pd.read_csv('FILTERED_MASTER_ANALYTICS_23_UNIS.csv', low_memory=False)

# Select the Features for Clustering
# Used outcomes, satisfaction, prestige, and custom value index
features = ['WORK', 'Q27', 'Prestige_Score', 'Course_Value_Index']

# Drop rows with missing values in these features to avoid errors
df_cluster = df.dropna(subset=features).copy()

# Scaling the Data
# K-Means is distance-based, so all features must be on the same 0-1 scale
scaler = StandardScaler()
scaled_features = scaler.fit_transform(df_cluster[features])

# Running K-Means
# Used K=4 to create 4 distinct "Strategic Categories"
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
df_cluster['Cluster_ID'] = kmeans.fit_predict(scaled_features)

# Interpret and Name the Clusters
# Used average scores of each cluster to assign a logical name
cluster_means = df_cluster.groupby('Cluster_ID')[features].mean()
print("Cluster Profiles:\n", cluster_means)

def name_clusters(cluster_id):
    mapping = {
        0: "Elite Academic Tier",      # High Prestige, High Outcomes
        1: "High-Value Career Hubs",   # High Employment, Moderate Prestige
        2: "Specialist & Niche",      # High Satisfaction, Moderate Outcomes
        3: "Developing/Entry Level"    # Lower scores across the 3 pillars
    }
    return mapping.get(cluster_id)

# Apply the names
df_cluster['Cluster_Category']                                          =
df_cluster['Cluster_ID'].apply(name_clusters)

# Save the final file
df_cluster.to_csv('FINAL_CLUSTERED_LONDON_COURSES.csv', index=False)

print("\nSuccess!          Final          file          saved          as
'FINAL_CLUSTERED_LONDON_COURSES.csv'")


# In[26]:


import pandas as pd
```

```
# Loaded a reference file that contained both UKPRN and University Names
for merging
reference_file = 'cleaned_london_enrollments_by_study_area_2020_2025.csv'
names_df = pd.read_csv(reference_file, low_memory=False)

# Create a clean Mapping (UKPRN -> University Name)
# Drop duplicates to ensure we have one unique name per UKPRN
ukprn_mapping             =             names_df[['UKPRN',             'HE
provider']].dropna().drop_duplicates('UKPRN')

# Load the Clustered Master file
master_file = 'FINAL_CLUSTERED_LONDON_COURSES.csv'
master_df = pd.read_csv(master_file, low_memory=False)

# Merge the names into the master file
# Used a left join to keep all the cluster data and just add the names
master_with_names   =   pd.merge(master_df,   ukprn_mapping,   on='UKPRN',
how='left')

# Reordered columns to put the Name at the front
cols = master_with_names.columns.tolist()
# Move 'HE provider' (index -1) to position 2
cols.insert(2, cols.pop(cols.index('HE provider')))
master_with_names = master_with_names[cols]

# Save the final version
master_with_names.to_csv('FINAL_CLUSTERED_LONDON_COURSES_WITH_NAMES.csv',
index=False)

print("University names successfully mapped and saved!")
```

APPENDIX B: UPDATED RISK REGISTER

Possible risks that could affect the successful completion of this project are:

- **Inconsistencies in Project Data**
  Due to the varied amount and sources of data required for this project, there is a high likelihood of inconsistencies – such as missing or duplicated values, hidden file markers or characters in CSVs and messy data – being present in the data. This can affect the outcome of the analysis, leading to wrong classifications and comparisons.

  To combat this, priority must be given to the data cleaning and pre-processing stage to ensure the data is balanced and normalized before any analysis can be carried out.

  Update: There were inconsistencies in the data, which meant several fields and records had to be dropped, including the degree qualification attributes (second class and third class attributes).

- **Poor Visualization of Data**
  The dashboard to be delivered at the end of the project will be displaying a wide array of information in various graphical formats. Thus, there is a risk that all that information will not be displayed efficiently, making it difficult for the user to make sense of the data.

  It is integral that while building the Tableau dashboard, the basic criteria to evaluate a visualization are kept in mind - expressiveness, effectiveness, consistency and importance order. This will ensure the dashboard serves it purpose of conveying the data correctly and effectively.

  Update: Using heuristic evaluation and Shneiderman's Eight Golden Rules of Interface Design, I was able to ensure all the visualizations are clear and easy to read.